

The Alberta Buck - Notes Mutual Decryptability (DRAFT v0.1)

Closing the Issuer-Binding Gap so Every BUCK Note Yields a Provable Receipt

Perry Kundert

2026-05-29



A monetary system can promise privacy or it can promise accountability; the Alberta Buck promises both at once, along a single seam. *Bulk* traceability is defeated – no chain-analytics firm can reconstruct who paid whom – while *bilateral* disclosure is preserved: the two parties to any payment always retain the cryptographic means to prove, under compulsion, exactly who the counterparty was. The load-bearing half of that promise is the **non-deniable-receipt invariant**: there must be no path by which someone pays BUCKs to another and keeps cryptographic deniability of the act. Concretely, a recipient who is compelled by a court, and who chooses to comply, must be able to produce a cryptographically sound receipt naming the payer's *decrypted, KYC-bound Identity*.

This document shows that BUCK Notes do not yet satisfy that invariant, defines it formally, and specifies the fix. The shipped spend path verifies the *depositor's* identity (`IdentityRegistry.verifySpendCP`), but the mint path verifies *nothing* about the *issuer's*: the per-leaf `issuerData` is folded into the commitment's `idHash` and never checked – exactly the "flavor-specific witness verification (Schnorr for A1/B1, ElGamal re-randomization well-formedness for A2)" that `mint.circom` marked "deferred to a later revision" and that was never re-introduced. A dishonest or colluding issuer can therefore mint a note that pays a recipient while leaving no recoverable payer Identity. The remedy is to make the *deferred approve handshake* symmetric and fully verified: bind a checked issuer identity at mint (a Schnorr binding for public issuers; a recipient-hiding re-encryption proof for the private A2 issuer), and add the `E_depositor_for_issuer` payload to B-note deposits. Each leg is stated as a theorem with an executable `py_ecc` check, and the circom / EVM formal-verification tooling that can mechanize the soundness arguments is catalogued. (Notes, Proofs)

Contents

1	The Non-Deniable-Receipt Invariant	4
1.1	Statement	4
1.2	Why the asymmetry between flavors	4
1.3	Why B2 (a private-issuer bearer note) is impossible	5
2	The Receipt as a Proof Chain	6
3	What the Shipped Implementation Binds	7
4	The Gap, Per Flavor	8
4.1	A2 – addressed, private issuer	8
4.2	A1 – addressed, public issuer	8
4.3	B1 – bearer, public issuer	8
4.4	Summary	8
5	The Fix: A Symmetric, Verified Handshake	9
5.1	Issuer half, bound at mint	9
5.2	Depositor half, at deposit	9
5.3	Enforcing bearer $\Rightarrow$ public issuer	10
6	Protocol Changes (Implementation Surface)	11
6.1	IdentityRegistry	11
6.2	Notes.sol	11
6.3	Circuits	11
6.4	Wallet (alberta_buck.wallet)	12
7	Provable Correctness	13
7.1	Theorems	13
7.2	Executable sanity checks (py_ecc on BN254)	13
8	Formal-Verification Approaches	15
8.1	The cryptographic mathematics	15
8.2	The circom circuits (soundness is the hard part)	15
8.3	The Solidity verifiers and Notes.sol	15
8.4	What "provably correct per step" buys	16
9	Test Plan	17
9.1	Positive	17
9.2	Negative (these must now fail, and did not before)	17
9.3	Cross-artifact	17
10	Phasing and Migration	18
11	Data Revealed Per Step, and Why Privacy Holds	19
11.1	Requirements and adversaries	19
11.2	The on-chain footprint of each step	19
11.3	Per-flavor data flow	19
11.3.1	B1 – bearer, public issuer	19

11.3.2	A1 – addressed, public issuer	20
11.3.3	A2 – addressed, private issuer (the hard case)	20
11.4	Why bulk-tracing and third-party privacy hold	20
11.5	Why mutual decryptability holds – the final data set	21
11.6	The colluding pair, and why enforcement must be at mint	22
12	The Required Mint SNARK Signal	23
12.1	What remains, and the decisions that shaped it	23
12.2	Why this preserves privacy	24
12.3	What the A2 tie buys, and what it does not	24
12.4	The A2 recipient-key coupling gap	24
12.4.1	Can it be closed now? The non-native-curve obstacle	25
12.4.2	The backstop: the gap degrades to KYC-traceability, not untraceability	26
12.4.3	Scope: the A2 coupling phase	26
12.4.4	The identity-targeted alternative: name the identity, not the account	27
12.5	Cost and migration	29
13	The EOA Transfer Mirror: Anonymity Is the Cost	30
13.1	The private->private EOA transfer (Alice -> Bob)	30
13.2	The A2 Note flow (Issuer I -> Recipient D)	30
13.3	The mapping	30
13.4	The structural diagnosis: the two anchors are never co-present	30
13.5	The finding: the cost is the anonymity	31
13.6	The recipient bears the burden – the load-bearing safeguard for A2	31
13.7	Decision: A1/B1 now, A2 reserved	32
13.8	Directions for future research	32
14	Implementation Status (Phase 1 + Phase 2 A2 binding + issuerMode + A2 leaf-tie shipped)	34
14.1	Phase 1 – shipped, wired, and tested	34
14.2	Phase 2 – A2 private-issuer binding (shipped, off-chain verifier + on-chain anchor)	35
14.3	Phase 2b – per-leaf issuerMode bound in the mint SNARK (shipped)	36
14.4	Phase 2c – A2 eIss leaf-tie (SHIPPED)	37
14.5	Decisions / deviations from the spec text above	37
14.6	Closed since Phase 1	38
14.7	Residual gaps (NOT done)	38
15	Remaining Work and Approaches	39
15.1	Cross-references and house conventions	39
16	Open Questions	40

STATUS (2026-06-10): historical – superseded. The non-deniable-receipt invariant defined here is stated in *alberta-buck-notes* ("The Non-Deniable-Receipt Invariant"); the per-step disclosure analysis survives, updated to the Identity-M surface, as *alberta-buck-notes-flow* "Privacy Audit: Data Revealed Per Step"; the theorems were modernised into *alberta-buck-proofs* Part IV (Theorems 7-12). The gaps this memo analysed are closed: the mint-side issuer bindings and – the final piece – the note $\langle \text{eEnc re-encryption tie} \rangle$ (`circuits/note_binding.circom`, `INoteBindingVerifier`). The fix-design, protocol-changes, test-plan, and phasing sections below reflect the pre-implementation state and are no longer maintained.

1 The Non-Deniable-Receipt Invariant

The Alberta Buck identity layer is built so that *every* BUCK transfer leaves both endpoints holding decryptable proof of the other's Identity. For a direct **transfer**, the bilateral **approve** handshake establishes this: each party re-encrypts their KYC-certified credential under the other's key and proves (Chaum-Pedersen) that the re-encryption carries the same Identity point M . BUCK Notes defer and reorganise that handshake through the commitment pool, and the invariant must survive the reorganisation.

1.1 Statement

Definition 1. Let an Identity be the KYC-certified point $M \in \mathbb{G}_1$ bound to a registered account by an issuer's Pointcheval-Sanders credential and stored (ElGamal-encrypted) as $E_{addr}[X] = (R_X, C_X) = (r_X G, M_X + r_X \cdot pk_X)$, with $pk_X = sk_X G$. A payment moves value v from a payer P to a recipient D .

Definition 2. (Non-deniable receipt.) A protocol satisfies the **non-deniable-receipt invariant** if for every accepted payment there exists a receipt $\mathcal{R}$ and an efficient public predicate `RcptVerify` such that:

- (**Completeness.**) The recipient D can compute $\mathcal{R}$ from data they hold plus public chain state, and `RcptVerify`($\mathcal{R}$) = 1 names a registered Identity M_P and value v .
- (**Soundness / non-frameability.**) No PPT adversary can produce $\mathcal{R}'$ with `RcptVerify`($\mathcal{R}'$) = 1 naming an Identity $M_{P'}$ that did not author the payment.
- (**No deniability.**) No PPT payer can cause an accepted payment to D for which no accepting receipt naming the payer exists – even in collusion with D .

The third clause is the operative one: it is the collusion-resistant form. A self-interested recipient who simply wants a record is protected by completeness alone; the invariant additionally forbids a *willing* pair from constructing an untraceable BUCK payment.

1.2 Why the asymmetry between flavors

Who can be named, and how, depends on what is known at mint time – which is exactly the axis that defines the note flavors:

- **B (bearer).** The depositor is unknown at mint, so the note cannot carry a recipient-targeted ciphertext. The *issuer* must therefore be a **public** Identity: their M is in the clear and

the recipient names them directly. A bearer note from an *encrypted* issuer (a "B2") is not merely undesirable but *impossible* – see *Why B2 is impossible* below – and the mint circuit + `Notes.mint` now enforce its absence.

- **A (addressed).** The issuer knows the recipient at mint and has already re-encrypted material the eventual depositor can open. The issuer may stay *private* (A2) by re-encrypting their *own* Identity under the recipient's key (`E_iss-for-rec`), which only the recipient can decrypt, or *public* (A1) by attaching it in the clear with a signature.

In all cases the recipient must end up holding a path to a *verified* issuer Identity. The next section shows the shipped code does not provide one.

1.3 Why B2 (a private-issuer bearer note) is impossible

The asymmetry above is not an implementation limitation – it is a theorem. The only party who could re-encrypt the issuer's Identity material so the *eventual* recipient can open it is the issuer, and the issuer does not know who that recipient will be.

Claim. A *meaningful* private-issuer bearer note (a "B2") cannot exist.

Proof. A bearer note's redeemer is unknown at mint: it is whoever comes to hold the bearer secret ρ . For that redeemer to name the issuer, the note must carry a binding to the issuer's *registered* Identity that the redeemer can decrypt and prove. The issuer creates this binding at mint – when the redeemer is unknown – so it must be openable under a key whose secret the eventual redeemer will hold. Two cases exhaust the options:

1. *The key is the redeemer's own registered key.* Impossible: the issuer does not know the redeemer and cannot target a key they have never seen.
2. *The key travels with the note* – a note-bound key whose secret is derived from ρ . Then it is openable by **every** holder, because holding ρ is exactly what makes one a holder; the issuer's Identity is revealed to the entire, freely-circulating bearer chain. That is no more private than B1 (a public issuer in the clear) – the issuer gains nothing by being "private".

A binding openable by *no one* fails decryptability outright. Hence the only binding that is both decryptable-by-the-redeemer and private-from-the-world requires a *designated* recipient – and a note with a designated recipient is an **addressed** (A) note. "Private issuer" and "bearer" are therefore mutually exclusive: B2 collapses into A2. ■

The contrast with A2 is the whole point. A2 reveals the issuer to exactly *one* party – the recipient the issuer chose – under the issuer's control; the re-encryption $E_{\text{iss-for-rec}}$ is openable only with sk_{rec} . B2 has no chosen party, so any binding it could carry is openable by everyone or by no one. (The earlier near-miss – publishing $T = r'pk_{\text{rec}}$, which leaked M_{iss} – was a *soundness* slip inside A2; B2 by contrast is a *structural* impossibility, independent of any binding construction.)

The protocol **enforces** the theorem rather than assuming it. The flavor set is exactly $\{A1, A2, B1\}$; there is no B2 codepoint. The mint circuit range-constrains $\text{flavor} \in \{1, 2, 3\}$ and projects the per-leaf `issuerMode` ($A2 \rightarrow \text{PRIVATE}$, $\{A1, B1\} \rightarrow \text{PUBLIC}$), which the Groth16 proof now *binds*; `Notes.mint` requires a registered public issuer + Schnorr for every PUBLIC-mode leaf. Because a bearer leaf is always PUBLIC-mode, a private issuer simply cannot mint a bearer note – the on-chain image of the proof above. See *The Required Mint SNARK Signal and Implementation Status*.

For the recipient to prove "I received v BUCKs issued by Identity M_P ", the receipt must chain four links, each independently checkable against public chain state:

- Links (a), (b), (d) are already sound in the shipped contracts. (b) holds because `cms []` live in the mint transaction’s calldata, which is immutable chain history; the recipient reveals `cm` to anchor it (accepting the `spend`↔`mint` linkage, which is the price of complying). (d) holds because the nullifier is a deterministic function of the opening and is emitted by `Notes.Spent / Notes.SpentA` alongside the ERC-20 `Transfer` from the pool.

The invariant lives or dies on link (c) – and (c) is precisely the step the implementation never enforces.

3 What the Shipped Implementation Binds

The spend side verifies the depositor's Identity; the mint side verifies nothing about the issuer's. This asymmetry is the whole bug.

Leg	Shipped check	Guarantee
Depositor (A-spend)	<code>IdentityRegistry.verifySpendCP(spender, recipient, E_n, pi)</code>	spender provably <i>is</i> the addressed M_{rec}
Depositor (B-spend)	none (knowledge of ρ is the authorization)	spender is <i>some</i> registered recipient
Issuer (any mint)	none – <code>idHash</code> folds <code>issuerData</code> via Poseidon-8, value never checked	none

Concretely, from the source:

- `circuits/spend_a.circom` constrains `idHash === Poseidon8(eNoteR.x, eNoteR.y, eNoteC.x, eNoteC.y, issuerData[0..3])`, where `issuerData` is a *private witness* holding A1 ($m_{iss}, \sigma_R.x, \sigma_R.y, \sigma_s$) or A2 (R_{iss}, C_{iss}) = $E_{iss-for-rec}$. The circuit *hashes it in* but never verifies the signature or the encryption.
- `circuits/spend.circom` (the shipped B-spend) leaves `flavor` unconstrained and binds *no* issuer material at all; `idHash` is opaque.
- `circuits/mint_batch.circom` public inputs are `[oldRoot, newRoot, nextLeafIndex, totalFace, cm]` with private `[flavor, v, rho, idHash, predicate, siblings]`. No issuer key, no signature, no ElGamal well-formedness. `mint.circom` says so in as many words: *"flavor-specific witness verification (Schnorr for A1/B1, ElGamal re-randomization well-formedness for A2) is deferred to a later revision."*
- `src/Notes.sol mint()` records `Minted(issuer = msg.sender, ...)` but ties the minting account to the leaves only through transaction authorship, never cryptographically to the committed `issuerData`.

4 The Gap, Per Flavor

4.1 A2 – addressed, private issuer

The opening carries $E_{\text{iss-for-rec}} = (R_{\text{iss}}, C_{\text{iss}})$, an ElGamal ciphertext the recipient decrypts with sk_{rec} to obtain M_{iss} . It is bound into `idHash` but **not** proven to re-encrypt the issuer’s *registered* credential. A malicious issuer commits a random $(R_{\text{iss}}, C_{\text{iss}})$: the note mints and spends cleanly, the recipient decrypts to a meaningless point, and `link(c)` yields no registered Identity. **The payer keeps full deniability.**

4.2 A1 – addressed, public issuer

The opening carries (m_{iss}, σ) . The Schnorr signature σ is never verified, and nothing forces m_{iss} to equal the *minting* account’s registered M . A malicious issuer can omit a valid σ (no receipt), or commit a *victim*’s m_{iss} and a forged σ (frame), and the note still works.

4.3 B1 – bearer, public issuer

`spend.circum` binds no issuer Identity whatsoever. The only issuer signal is `msg.sender` in the `Minted` event. A receipt is therefore constructible *only if* the issuer minted from a **public** Identity – and the contract cannot enforce that, because `cms[]` are opaque and it cannot tell a bearer leaf from an addressed one. A bearer-spendable note minted from an *encrypted* Identity is unnameable. **The payer keeps full deniability.**

4.4 Summary

In every flavor the issuer Identity is, at best, *committed* but never *verified*; in B1 it is not even committed. The depositor half of the handshake is checked (`verifySpendCP`); the issuer half is missing. The fix is to add it.

5 The Fix: A Symmetric, Verified Handshake

Restore the symmetry the "deferred approve handshake" framing always implied: the *mint* is the issuer's half of approve and must be *verified*, exactly as the *deposit* verifies the depositor's half.

5.1 Issuer half, bound at mint

At mint, the issuer is `msg.sender` – *not* anonymous (they authored the tx). So the issuer's registered credential $E_{\text{addr}}[\text{issuer}]$ may be read as a public input; only the *recipient* (for A2) must stay hidden.

- **Public issuer (A1, B1).** Require `msg.sender` to be `isPublicIdentity`, and verify a Schnorr signature by their registered key pk_{iss} over the batch commitment $H_{\text{batch}} = \text{keccak}(\text{cms})$. Bind $m_{\text{iss}} = M_{\text{iss}}$ (the in-the-clear registered Identity) into `idHash`. Because the registered Identity of a public account is in the clear, link (c) reduces to a registry read plus the verified signature.
- **Private issuer (A2).** The issuer proves `E_iss-for-rec` re-encrypts their *own* registered M_{iss} under the recipient's key pk_{rec} – the `verifyApprove` relation with `sender = issuer`, `spender = recipient`. But `verifyApprove` as shipped takes pk_{rec} as a verifier input and so *names the recipient*, which A2 must not do. Two recipient-hiding realisations:

1. **In-SNARK (mint circuit).** Extend `mint_batch.circom` so each A2 leaf proves, with pk_{rec} and M_{iss} as private witnesses:

$$pk_{\text{iss}} = sk_{\text{iss}}G, \quad M_{\text{iss}} = C_{\text{iss}}^{\text{reg}} - sk_{\text{iss}}R_{\text{iss}}^{\text{reg}}, \quad E_{\text{iss-for-rec}} = (r'G, M_{\text{iss}} + r'pk_{\text{rec}}),$$

with $E_{\text{addr}}[\text{issuer}] = (R_{\text{iss}}^{\text{reg}}, C_{\text{iss}}^{\text{reg}})$ a *public* input and pk_{rec} the same key that re-randomises the note ciphertext E_{note} . Cost: a handful of in-circuit $\mathbb{G}_1$ scalar muls per A2 leaf – the ~10x the original deferral feared, but bounded and paid only on A2 mints.

2. **Recipient-blinded on-chain CP.** Keep verification on-chain (EIP-196, cheap) but blind pk_{rec} : the issuer proves the re-encryption relation against a *Pedersen commitment* to pk_{rec} that also appears (identically committed) in E_{note} , so the verifier never learns pk_{rec} . This is a new sigma protocol (Section *Provable Correctness*), avoiding in-SNARK $\mathbb{G}_1$ arithmetic at the cost of a bespoke proof.

The in-SNARK route (1) is the recommended default: it reuses the trusted setup already in place for the batch mint and keeps the on-chain mint a single verifier call.

5.2 Depositor half, at deposit

- **A1, A2.** Already shipped: `verifySpendCP` proves the spender is the addressed M_{rec} . The issuer, who chose M_{rec} at mint, already holds the symmetric knowledge.
- **B1.** Add `E_depositor_for_issuer` to the spend: the depositor re-encrypts *their* registered M_{dep} under the public issuer's pk_{iss} , with a `verifyApprove`-style CP proof (`sender = depositor`, `spender = issuer`, both public-or-named here so no blinding is needed), emitted in the `SpentB` event. This closes the symmetric direction – the public bearer issuer can identify who cashed their note.

5.3 Enforcing bearer $\Rightarrow$ public issuer

The contract cannot read `flavor` from an opaque `cm`. Make the mint flavor-aware for the binding: the mint circuit exposes, per leaf, a public `issuerMode` in `{PUBLIC, PRIVATE}` and proves the matching binding (Schnorr-bound public M , or in-SNARK re-encryption). `Notes.mint` requires every `PRIVATE` leaf to be an A-flavor (addressed) leaf – a bearer (B) leaf with `issuerMode = PRIVATE` is unsatisfiable – and requires `msg.sender` to be `isPublicIdentity` whenever any `PUBLIC` leaf is present.

DRAFT

6 Protocol Changes (Implementation Surface)

6.1 IdentityRegistry

```
// Public-issuer binding: Schnorr by msg.sender's registered key over H_batch.
function verifyIssuerSchnorr(address issuer, bytes32 hBatch, SchnorrProof calldata s)
    external view returns (bool);

// Private-issuer binding (A2), recipient-blinded sigma variant (option 2):
// proves E_iss_for_rec re-encrypts E_addr[issuer]'s M under a *committed* pk_rec
// that the mint SNARK also binds to E_note, without revealing pk_rec.
function verifyIssuerCPBlinded(address issuer, ElGamalCT calldata eIssForRec,
    bytes32 pkRecCommit, IssuerCPPProof calldata pi)
    external view returns (bool);

// B-note depositor->issuer (mirror of verifyApprove, both parties named):
function verifyDepositorForIssuer(address depositor, address issuer,
    ElGamalCT calldata eDepForIss, CPPProof calldata pi)
    external view returns (bool);
```

Under the in-SNARK route the A2 binding is discharged inside the mint verifier and `verifyIssuerCPBlinded` is unnecessary; the registry then only gains `verifyIssuerSchnorr` and `verifyDepositorForIssuer`.

6.2 Notes.sol

- `mint(...)` gains per-leaf `issuerMode[]` (public input of the mint SNARK) and, for public leaves, a `SchnorrProof` over `keccak(cms)`; it requires `msg.sender` public when any public leaf is present, and rejects `PRIVATE+bearer` leaves. The A2 re-encryption binding rides inside the mint proof (option 1) or is checked per leaf via `verifyIssuerCPBlinded` (option 2).
- `spend(...)` (B) gains `(E_depositor_for_issuer, CPPProof)`, calls `verifyDepositorForIssuer(msg.sender, issuer, ...)`, and emits `SpentB(nullifier, face, recipient, eDepForIss)`. The issuer address is recovered from the leaf's public-issuer binding revealed at `spend`.
- `spendACP(...)` (A) is unchanged on the depositor side; the issuer guarantee now comes for free from the mint-time binding.

6.3 Circuits

- `mint_batch.circom`: add the per-leaf `issuerMode` public signal and, for A2 leaves, the re-encryption constraints above (option 1). For public leaves, constrain `idHash`'s `issuerData` prefix to equal the registered public `M` so the in-clear identity is the one bound.
- `spend.circom` (B): constrain `flavor == B1` and reveal enough of the leaf's public-issuer binding to recover the issuer address on-chain.
- `spend_a.circom`: unchanged (the A2 issuer binding moved to mint).

6.4 Wallet (`alberta_buck.wallet`)

- Mint: construct the issuer Schnorr proof (public) or the A2 re-encryption witness (private), mirroring `notes.id_hash_a1/a2`.
- Spend (B): construct `E_depositor_for_issuer` and its CP proof.
- Receipt: implement `RcptVerify` as a standalone verifier (opening + mint-tx reference + spend-tx reference + issuer recovery) so a recipient can produce and a third party can check the receipt entirely off-chain from chain data.

DRAFT

7 Provable Correctness

We extend the companion *Proofs* document (Theorems 1-11) with the issuer-binding results. Each is accompanied – in the proofs document’s house style – by an executable `py_ecc` sanity check on BN254.

7.1 Theorems

Theorem 3. (Issuer-binding completeness.) *An honestly generated public-issuer Schnorr proof, and an honestly generated A2 re-encryption proof (in-SNARK or blinded), verify.*

Theorem 4. (Issuer-binding soundness.) *If `verifyIssuerSchnorr` accepts, the batch commitments were authorised by `issuer`’s registered key. If the A2 re-encryption proof accepts, $E_{iss-for-rec}$ decrypts under sk_{rec} to the issuer’s registered M_{iss} (the same M certified by the issuer’s PS credential – Theorem 6). Reduction: special soundness of the underlying sigma protocol (two accepting transcripts sharing commitments extract sk_{iss}, r'), as in Theorems 4, 6; or Groth16 knowledge-soundness for the in-SNARK route.*

Theorem 5. (Receipt completeness.) *For every note accepted by the patched `mint` and redeemed by `spend` $\neq$ `spendACP`, the recipient can compute $\mathcal{R}$ with $RcptVerify(\mathcal{R}) = 1$ naming a registered Identity. Proof: links (a),(b),(d) hold as before; (c) now holds by issuer-binding soundness (public: registry $M + Schnorr$; private: decrypt $E_{iss-for-rec}$, validity guaranteed by the accepted binding).*

Theorem 6. (Receipt soundness / non-frameability.) *No PPT adversary produces an accepting $\mathcal{R}'$ naming $M_{P'}$ that did not author the payment. Reduction: a forged public receipt breaks Schnorr EUF-CMA; a forged private receipt breaks re-encryption soundness (Theorem 7) or the binding hash (collision resistance of Poseidon-8 in `idHash`).*

Theorem 7. (No deniability under collusion.) *No payer-recipient coalition can produce an accepted payment with no accepting receipt: `mint` rejects any leaf lacking a valid issuer binding, so every spendable leaf carries one, and the binding is recoverable by the holder of the opening.*

Theorem 8. (Recipient privacy preserved.) *The A2 issuer binding is zero-knowledge in pk_{rec} and M_{iss} : the in-SNARK route reveals only the Groth16 proof; the blinded route reveals only a hiding Pedersen commitment to pk_{rec} . No observer learns the recipient. Reduction: HVZK of the sigma protocol (Theorem 5) / Groth16 zero-knowledge.*

Corollary 9. (Mutual decryptability.) *Combining the issuer half (above) with the depositor half (`verifySpendCP` for A; `verifyDepositorForIssuer` for B), both parties can recover the other’s registered Identity, and each can prove it under compulsion. This is the Notes specialisation of the BUCK mutual-decryptability invariant (Proofs Theorem 11), now enforced rather than assumed.*

7.2 Executable sanity checks (py_ecc on BN254)

Mirroring the proofs document, each theorem ships a small witnessed check that runs in CI alongside the existing `alberta_buck.wallet` vector generation:

- **Issuer Schnorr:** sign H_{batch} under sk_{iss} , assert verify; assert a tampered H_{batch} or wrong key fails.

- **A2 re-encryption:** build $E_{\text{addr}}[\text{issuer}]$, decrypt to M_{iss} , form $E_{\text{iss-for-rec}}$ under pk_{rec} , run the prover/verifier; assert the recipient's sk_{rec} -decryption equals M_{iss} ; assert a random $E_{\text{iss-for-rec}}$ is rejected (soundness witness) and that the transcript hides pk_{rec} (two runs with distinct pk_{rec} are indistinguishable to the verifier given the commitment).
- **Receipt:** assemble $\mathcal{R}$ from a synthetic mint+spend and assert $\text{RcptVerify} = 1$; assert a frame attempt (swap M_P) fails.

These vectors double as the canonical inputs the Solidity and circom tests load via `vm.readFile`, keeping the on-chain verifier, the circuit, and the reference math in lock-step (the pattern already used for `verifySpendCP`).

DRAFT

8 Formal-Verification Approaches

"Every step provably correct" spans three artifact classes – the sigma-protocol mathematics, the circom circuits, and the Solidity verifiers – each with its own tractable tooling.

8.1 The cryptographic mathematics

- **Hand proofs** + `py_ecc`: the existing *Proofs* methodology – completeness, special-soundness (2-extractability), and HVZK arguments, each pinned by an executable BN254 check. Low-tech, high-confidence, already in the repo.
- **EasyCrypt**: a proof assistant purpose-built for *game-based* cryptographic proofs; the sigma-protocol soundness and ZK lemmas can be mechanised as reductions, giving machine-checked versions of Theorems above.
- **CryptoVerif** (computational) or **ProVerif** (symbolic): protocol-level analysis of the full handshake (mint binding + deposit binding) against the non-deniability and recipient-privacy goals stated as queries.

8.2 The circom circuits (soundness is the hard part)

The danger in arithmetic circuits is *under-constraint* – a witness slot that the prover can set freely, silently widening soundness (the exact failure mode of the unverified `issuerData`). Tooling that targets this directly:

- **Picus** (Veridise): automated detection of under-constrained signals; run it on the patched `mint_batch.circom` to prove the new issuer-binding signals are *determined* by the public inputs.
- **Circomspect** (Trail of Bits): static analysis / linting for common circom pitfalls (unconstrained signals, `<--` without `<=`, `=` signal-vs-constraint confusion).
- **ecne** (0xPARC): proves R1CS systems compute deterministic functions of their inputs – a soundness witness for the binding constraints.
- **Coda** / **circom-to-Coq** efforts: full functional-correctness extraction to a proof assistant for the security-critical templates (the re-encryption gate).
- Discipline: keep the security-critical constraints (the $\mathbb{G}_1$ relations) in small, separately-audited sub-templates with explicit `Num2Bits` range checks, as the value-conservation bound already does.

8.3 The Solidity verifiers and Notes.sol

- **Halmos** (a16z): symbolic-execution property tests in Solidity – assert, for all inputs, e.g. "mint reverts unless every leaf carries a valid issuer binding" and "spend emits a recoverable issuer for every accepted note."
- **Kontrol** / **KEVM** (Runtime Verification): K-framework formal proofs over EVM bytecode for the highest-assurance properties (the binding-or-revert invariant, nullifier non-reuse).

- **Certora Prover:** CVL rules for state invariants (`noteFaceSum` conservation, "no accepted spend without a prior valid mint binding").
- **solc SMTChecker:** lightweight, in-compiler assertions and overflow checks on the new code paths.
- **Foundry:** fuzz + stateful invariant testing as the always-on backstop; negative tests below are expressed here first.

8.4 What "provably correct per step" buys

The chain of guarantees is: `=py_ecc=`/EasyCrypt prove the *math*; Picus/ecne prove the *circuit* faithfully and exhaustively enforces that *math*; Halmos/Kontrol prove the *contract* admits no spend whose mint lacked the binding. Composed, no accepted Note payment can exist without a recoverable, sound receipt – the invariant, mechanically checked end-to-end.

DRAFT

9 Test Plan

9.1 Positive

- A1/A2/B1 honest mint + spend; recipient runs `RcptVerify` and a third party re-checks it – all return 1 naming the correct registered Identity.
- A2 recipient privacy: two mints to distinct recipients are observationally indistinguishable on chain.

9.2 Negative (these must now fail, and did not before)

- A2 issuer commits a random `E_iss-for-rec`: `mint reverts` (binding fails).
- A1 issuer commits a victim's m_{iss} / a forged σ : `mint reverts`.
- Bearer leaf with `issuerMode = PRIVATE`, or any public leaf from a non-`isPublicIdentity` minter: `mint reverts`.
- Recipient attempts a forged receipt naming a non-issuer: `RcptVerify = 0`.

9.3 Cross-artifact

- The `py_ecc` vectors drive both the circom witness tests and the Solidity verifier tests, asserting byte-identical `idHash` / proof encodings (the established `alberta_buck.wallet -> vm.readFile` pattern).
- Picus run in CI gates merges on "no new under-constrained signals."

10 Phasing and Migration

1. **Public issuers first (A1, B1).** Schnorr binding + `isPublicIdentity` enforcement + `verifyDepositorForIssuer` for B. No new circuit setup; pure contract + wallet work. Closes the gap for the public-issuer flavors, which are the audit-sensitive payroll / cashier-cheque cases.
2. **Private issuer (A2).** The in-SNARK re-encryption binding: extend `mint_batch.circom`, re-generate the per-N verifiers and trusted setup, wire `issuerMode`. The larger lift; ship behind a governance-set verifier slot so the new mint verifier can be rotated in (`Notes.setMintVerifier`).
3. **Mechanisation.** Land Picus/Halmos in CI; mechanise the sigma proofs in EasyCrypt as time allows.

Because nothing is deployed yet (all R&D), there is no migration window to honour: `Notes.mint` is **gated-only**. Every mint goes through a binding overload – PUBLIC (a registered public issuer + Schnorr) or PRIVATE (a registered private issuer + the per-leaf A2 leaf-tie binding) – and there is no unbound path, so an unnameable note cannot be created and every minter is a registered Identity. The invariant is therefore *enforced unconditionally* (a UNVERIFIED ISSUER receipt can now arise only from the A2 recipient-key coupling gap, not from an unbound mint). (An earlier draft kept legacy unbound overloads behind a `bindingRequired` cutover flag for a hypothetical deployed-contract migration; with nothing deployed that was obsolete machinery and was removed in favour of the clean gated-only surface.)

11 Data Revealed Per Step, and Why Privacy Holds

The whole construction has to thread one needle: publish enough that each party can *name* the other under compulsion, while publishing so little that no *third party* can name either, or link payments in bulk. This section walks every step of every flavor, tabulates exactly what each participant reveals to whom, and argues – step by step – that the two privacy requirements survive and the mutual-decryptability requirement is met by the data each party is left holding. The reasoning is grounded in a concrete near-miss: the first A2-binding draft leaked the private issuer’s Identity, and showing *why* the shipped version does not is the clearest way to see the invariant.

11.1 Requirements and adversaries

Definition 10. *Three properties, against three adversaries:*

- **(Bulk-tracing resistance.)** Mallory, a passive observer with the full chain history and unbounded compute but no party’s secrets, cannot reconstruct who paid whom, how much, or link a mint to its spend(s).
- **(Third-party non-disclosure.)** Nothing on chain reveals either party’s KYC-bound Identity point M to a non-participant – not even to the counterparty’s counterparty, and not in aggregate across many payments.
- **(Mutual decryptability.)** Each of the two participants, from data they hold plus public chain state, can produce a sound receipt naming the other’s registered M – and a colluding pair cannot arrange a payment for which no such receipt exists (the non-deniable-receipt invariant).

The first two are *confidentiality* against outsiders; the third is *accountability* between insiders. They pull in opposite directions, so every datum published is justified below against all three.

11.2 The on-chain footprint of each step

What actually lands on chain, and what stays off it:

Step	Published on chain	Kept off chain
register / bindContract	addr, pk, E_addr (ElGamal ct of M under own pk), isPublicIdentity	m, M, sk
mint (any flavor)	Minted(issuer, totalFace, startIndex, count, newRoot); cms[] in calldata (opaque Poseidon)	per-leaf opening
mint public (A1/B1)	+ IssuerBound(issuer, newRoot); one Schnorr over keccak(cms)	–
mint private (A2)	+ IssuerReencBound(issuer, newRoot, count); per-leaf (Q, U, T_hat, A_1..A_5)	pk_{rec}, M_{iss}, r'
B-spend	Spent(nf, face, recipient) or SpentB(nf, face, recipient, issuer, eDepForIss)	$\rho, idHash, op$
A-spend	SpentA(nf, face, recipient); revealed E_n; CP-DLEQ (e, s, T_1, T_2)	$\rho, idHash, sk_d$

Two facts do the heavy lifting throughout. First, cms[] are Poseidon commitments: a leaf reveals *nothing* about (*flavor, v, $\rho, idHash, predicate$*) without ρ . Second, the spend nullifier $nf = \text{Poseidon}_3(\rho, idHash, tag)$ is a PRF output over secret material, so it is unlinkable to any cm without the opening.

11.3 Per-flavor data flow

11.3.1 B1 – bearer, public issuer

- **Mint.** The issuer is a registered isPublicIdentity account, so its M_{iss} is attested in the clear off chain. On chain: opaque cms[], Minted, and one Schnorr over keccak(cms) (IssuerBound). Mallory sees "public account A escrowed totalFace into N opaque leaves and signed the batch." No per-leaf value, recipient, or even count of *live* vs padding leaves.

- **Off-chain delivery.** The bearer note carries the opening plus (m_{iss}, σ_{iss}) – the issuer’s public Identity and a signature. Anyone who holds the note learns the issuer (intended: a cashier’s cheque names its bank).
- **Spend.** The depositor publishes nf , **face**, **recipient**. Optionally (the 8-arg path) **SpentB** additionally carries $E_{dep-for-iss}$, a re-encryption of the *depositor’s* registered M_{dep} under the public issuer’s pk_{iss} , with a CP proof – so the issuer can later name who cashed the note.
- **Final holdings.** Recipient holds M_{iss} in the clear (names the issuer). Issuer, reading **SpentB**, decrypts $E_{dep-for-iss}$ with sk_{iss} to recover M_{dep} (names the depositor). Mutual.

11.3.2 A1 – addressed, public issuer

- **Mint.** As B1 (public issuer, Schnorr-bound batch), but the leaf is addressed: $idHash = \text{Poseidon}(E_{note}, m_{iss}, \sigma)$ with E_{note} a re-randomisation of the recipient’s registered credential. Still opaque on chain.
- **Off-chain delivery.** The recipient scans candidate notes by trial-decrypting E_{note} with sk_{rec} ; a match reveals it is theirs. They learn M_{iss} from the in-clear payload.
- **Spend.** **spendACP** reveals E_n (the note ciphertext) and a CP-DLEQ proving sk_{dep} decrypts both E_n and the spender’s registered E_{addr} to the same M – so **msg.sender** is the addressed recipient. Publishes **SpentA**(**nf**, **face**, **recipient**).
- **Final holdings.** Recipient names the issuer (public $M_{iss} + \text{Schnorr}$); issuer knew the recipient at mint (they chose pk_{rec}). Mutual.

11.3.3 A2 – addressed, private issuer (the hard case)

Both parties are encrypted-Identity; this is where every published byte is scrutinised.

- **Mint.** `Notes.mint(..., A2Binding[])`. For each A2 leaf the issuer publishes $E_{iss-for-rec} = (R_i, C_i) = (r'G, M_{iss} + r'pk_{rec})$ and the recipient-blinded binding (Q, U, T_hat, A_1..A_5, responses), verified by **verifyIssuerReenc**. **IssuerReencBound** anchors the count.
- **Off-chain delivery.** The issuer sends the opening, E_{note} , $E_{iss-for-rec}$, the binding, and the un-blinding scalar γ . The recipient scans E_{note} , then decrypts $E_{iss-for-rec}$ with sk_{rec} to recover M_{iss} .
- **Spend.** **spendACP** exactly as A1 (reveals E_n , CP-DLEQ, **SpentA**).
- **Final holdings.** Recipient holds M_{iss} (decrypted) and the binding + γ , which together prove it is the issuer’s *registered* Identity (below). The issuer, having addressed the note, knows M_{rec} . Mutual.

11.4 Why bulk-tracing and third-party privacy hold

Mint is opaque; spend is unlinkable to mint. At mint Mallory sees opaque `cms[]` and a `totalFace`; she cannot read any v_i , recipient, or flavor (Poseidon hiding). At spend she sees a fresh nf , a **face**, and a **recipient**; nf is a PRF over $(\rho, idHash)$, so inverting it to a specific `cm` is pre-image resistance. No field of a spend points back to a mint. The only residual linkage surfaces

are the two that *must* be public for the ERC-20 to pay out – **face** and **recipient** – mitigated by fixed denominations and relayed payouts, orthogonal to this document.

Identities never appear on chain – the load-bearing check. A registered E_{addr} is an ElGamal *ciphertext* of M ; reading it yields nothing without sk . The danger is in the *bindings*, which manipulate M and pk_{rec} algebraically. Concretely, the A2 binding must publish enough to prove " $E_{iss-for-rec}$ re-encrypts the issuer's registered M_{iss} under the recipient's pk_{rec} " without revealing either point. This is exactly where the first draft failed:

Near-miss (rejected draft). The binding published $T = r' * pk_{rec}$.

But the leaf is $C_i = M_{iss} + r' * pk_{rec}$, so ANY observer computes

$$M_{iss} = C_i - T.$$

At mint `msg.sender` IS the issuer, so this publicly bound the issuer's address to its Identity -- collapsing A2 to A1 and breaking BOTH third-party non-disclosure and bulk unlinkability (every A2 mint by the same issuer now carries the same recoverable M_{iss}).

The shipped binding blinds it: it publishes $\hat{T} = r'pk_{rec} + \gamma G$ for a fresh γ . Now $C_i - \hat{T} = M_{iss} - \gamma G$, a uniformly random offset from M_{iss} – so $\hat{T}$ reveals nothing about M_{iss} . The recipient's key pk_{rec} is likewise hidden: it appears only inside $Q = pk_{rec} + \beta H$ (perfectly hiding for uniform β) and $U = r'H, \hat{T}$ (uniform). Soundness is undisturbed because γ cancels in the verification relations (L3, L5): the proof still forces $C_i = M_{iss}^{reg} + r'pk_{rec}$. The commitments $A_1..A_5$ are zero-knowledge. So an A2 mint's on-chain footprint is, to Mallory, a vector of uniformly-distributed group elements: it discloses neither identity nor recipient, and two A2 mints by the same issuer to the same recipient are indistinguishable from independent draws.

What issuerMode will and will not leak. Closing the residual gaps (below) requires the mint SNARK to expose, per leaf, a public *issuerMode* $\in \{PUBLIC, PRIVATE\}$. That reveals the *flavor class* of each leaf – not its value, recipient, or identity. Flavor-class leakage is acceptable: it is the same information `isPublicIdentity(msg.sender)` already exposes for the batch, and it does not narrow the identity or amount anonymity sets. The A2 binding values stay blinded regardless.

CP-DLEQ at spend reveals the spender, not the issuer. `spendACP` publishes E_n and a CP-DLEQ. E_n is a re-randomised ciphertext (uniform), and the DLEQ binds `msg.sender`'s *registered* key – i.e. it identifies the *spender to themselves* (they are acting), exactly as an EOA sender is the `msg.sender` of their own transfer. It reveals nothing about the issuer: the issuer's M_{iss} is recovered only off chain, by the recipient decrypting $E_{iss-for-rec}$, and disclosed only in a compelled receipt.

11.5 Why mutual decryptability holds – the final data set

For each flavor, list the data each party ends holding and the predicate it satisfies. The receipt verifier (`alberta_buck.wallet.receipt / verify_receipt`) is the public predicate `RcptVerify`.

Flavor	Recipient names issuer via	Issuer names depositor via
B1	in-clear $M_{iss} + \text{batch Schnorr (verifyIssuerSchnorr)}$	SpentB : decrypt $E_{dep-for-iss}$ with sk_{iss}
A1	in-clear $M_{iss} + \text{batch Schnorr}$	chose pk_{rec} at mint (symmetric knowledge)
A2	decrypt $E_{iss-for-rec} \rightarrow M_{iss}$; binding + γ prove it is the registered M_{iss}	chose pk_{rec} at mint

The A2 row is the subtle one, and the **coupling** is what closes it. The recipient holds three facts: (i) the binding proves $C_i = M_{iss}^{reg} + r'pk_{rec}$ with M_{iss}^{reg} the issuer's registered Identity (and, via γ , $M_{iss}^{reg} = C_i - \hat{T} + \gamma G$, a value the receipt verifier recomputes); (ii) their verifiable decryption

proves $M_{\text{named}} = C_i - sk_{\text{rec}}R_i = C_i - r'pk_{\text{rec}}$; (iii) both equal the named point M . Subtracting, $r'pk_{\text{rec}} = r'pk_{\text{rec}}$ forces pk_{rec} = the key in Q – the $E_{\text{note}} \leftrightarrow Q$ coupling – so the recovered M is provably (a) the issuer's registered Identity and (b) addressed to *this* recipient. No new proof was needed: the coupling falls out of composing the mint binding with the recipient's own decryption. This is why the A2 receipt now reads **VALID** rather than **UNVERIFIED ISSUER**.

11.6 The colluding pair, and why enforcement must be at mint

Completeness (an honest recipient *can* produce a receipt) is met off chain. The third clause – no deniability even when *payer and payee collude* – needs the binding to be *unavoidable*: there must be no way to create a spendable leaf lacking it. Off-chain verification cannot give this (a colluding recipient simply accepts a binding-less note), so the check must be enforced on chain at mint. `verifyIssuerReenc` is called from `Notes.mint`, but it verifies the bindings the issuer *supplies*. Phase 2c's leaf-tie now (a) ties each $E_{\text{iss-for-rec}}$ to a specific committed leaf and (b) requires *every* A2 leaf to carry one – the SNARK exposes per-leaf `eIss`, which `Notes.mint` passes as the proof's public input. This upgrades the A2 guarantee from "sound for an honest issuer" to "no leaf unbound, no binding floating." It does **not** reach full collusion-resistance: the binding does not force `eIss`'s key to be the recipient's, so a colluding pair can still encrypt `eIss` under a throwaway key (*The A2 recipient-key coupling gap*, above). Closing that needs the in-SNARK $E_{\text{note}} \leftrightarrow eIss$ coupling, the recommended next phase.

DRAFT

12 The Required Mint SNARK Signal

Three of the residual gaps (bearer-from-non-public, the B-spend issuer tie, and A2 leaf-binding + completeness) converge on one circuit change: the batch-mint circuit must expose, per leaf, a verified public `issuerMode` in `{PUBLIC, PRIVATE}` and the leaf's issuer payload, so `Notes.mint` can gate on them. The `issuerMode` half is now **shipped** (see below and *Implementation Status*); the per-leaf issuer-payload exposure is the remaining increment on the same foundation.

12.1 What remains, and the decisions that shaped it

With `issuerMode` shipped, two items from the original convergence list remain. This session's design pass **changed how both are done**, under one governing constraint: on-chain EVM cost is paramount, while off-chain wallet compute is effectively free (the wallet has full chain history, Holochain bulk-data access, and ample CPU; expensive checks before-mint and on-receipt- before-deposit are fine).

- **Per-leaf issuerMode** (public output) – **SHIPPED** (b14e0f0, 584bb52, 68e5b55). `mint_batch.circom` range-constrains `flavor` \in `{A1,A2,B1}` and emits `issuerMode[i]` as the deterministic projection ($A2 \rightarrow \text{PRIVATE}$, $\{A1, B1\} \rightarrow \text{PUBLIC}$); the regenerated per-N Groth16 verifiers *bind* it (public-signal vector [`issuerMode[0..N]`, `oldRoot`, `newRoot`, `nextLeafIndex`, `totalFace`, `cm[0..N]`], arity $2N+4$). `Notes.mint` gates: every `PUBLIC` leaf needs `msg.sender isPublicIdentity` + a Schnorr over `keccak(cms)`; every `PRIVATE` leaf needs an A2 binding. Because a bearer leaf always projects to `PUBLIC` and there is no B2 codepoint, a bearer-from-non-public mint is unprovable – the on-chain enforcement of *Why B2 is impossible*.
- **Public-leaf identity binding – decided: OFF-CHAIN, no circuit/contract change.** The original plan constrained the leaf's committed m_{iss} to equal the minter's registered M in-circuit. Dropped: a public issuer is already named soundly by `msg.sender` + the Schnorr batch binding, and the wallet finds the mint tx from full history (search `Notes.mint` calldata for the `cm`) $\rightarrow$ `msg.sender` $\rightarrow$ registry M . The committed m_{iss} is redundant for naming, so the receipt names `msg.sender`'s M and ignores the committed scalar; there is nothing to enforce on chain. Bonus: **not** exposing m_{iss} denies a bulk observer a shared scalar by which to group one public issuer's notes.
- **A2 leaf $\leftrightarrow$ binding tie – decided: RAW exposure via a DEDICATED A2 mint circuit (SHIPPED, Phase 2c).** Expose each A2 leaf's $E_{iss-for-rec} = (R_i, C_i)$ as four public outputs and have `Notes.mint` match them against the `eIss` in each binding. *How the match is realised* (refined in implementation): rather than a contract-side field-compare, `Notes.mint` passes the bindings' `eIss` as the A2 proof's public input, so the Groth16 accept itself enforces "committed leaf's `eIss` = binding's `eIss`" – the equality is a SNARK constraint, not EVM code. The only contract-side `eIss` check is the canonical `< FIELD_R` bound (the verifier rejects public inputs $\geq$ the scalar field anyway). Two sub-decisions stand:
 1. **Raw, not hash-bound.** A SNARK-friendly digest (Poseidon) is EVM-hostile: matching $H(\text{binding.eIss}) = \text{tag}[i]$ would force the contract to recompute Poseidon *on chain* ($\sim$ tens of K gas/leaf, plus a hashing dependency `Notes` does not have today). Raw `eIss` + a field compare is cheaper and simpler, and `eIss` is already on chain in the binding calldata, so raw exposure leaks nothing new. (A SNARK-friendly hash is cheap in-circuit but dear on-chain; an EVM-friendly hash (keccak) is the reverse – there is no hash cheap on both sides, so for a value both must agree on, expose it raw.)

2. **A dedicated circuit, not the shared one.** A Groth16 circuit has fixed public-input arity, so adding `eIss` to `mint_batch` would make *every* public/bearer batch pay for $4N$ all-zero inputs. Instead `circuits/mint_batch_a2.circom` is a separate family used only by the private-mode mint: it constrains `flavor = A2`, opens `idHash` to `Poseidon8(eNote, eIss)` (the V2 A2 layout), and exposes `eIss` (layout `[eIss[0..N]]` (4 each), `oldRoot`, `newRoot`, `nextLeafIndex`, `totalFace`, `cm[0..N]`), arity $5N+4$). Public/bearer batches keep using `mint_batch` unchanged and pay nothing for the A2 machinery.

12.2 Why this preserves privacy

In `mint_batch_a2`, `eNote` (the note ciphertext the A-spend later reveals) stays a PRIVATE witness, so `mint` ↔ `spend` remain unlinkable; only `eIss` – already public in the A2 binding – is exposed. `issuerMode` leaks only flavor-class. Amounts and identities stay hidden; value-conservation and the Merkle relations are unchanged.

12.3 What the A2 tie buys, and what it does not

The count-only check `Notes.mint` shipped before Phase 2c verifies one binding *per private leaf* by **count**, so a willing issuer+recipient pair can submit N copies of one binding while committing N different `eIss` – only the matching leaf is bound, the rest carry committed `eIss` with no verified binding at all. The leaf-tie closes exactly this: the bindings' `eIss` are the A2 circuit's public inputs, so a Groth16 accept proves each committed leaf carries its own verified binding – no leaf unbound, no binding floating to a different leaf. It *must* be on-chain – an off-chain receipt check is skippable by a colluding recipient.

What it does NOT buy: the recipient-key coupling. The leaf-tie binds each committed leaf to a verified re-encryption of the issuer's *registered* Identity – but `verifyIssuerReenc` binds `eIss` to the key committed in the proof's `Q`, which the **issuer** chooses (it is *not* proven to be the addressed recipient's registered key; see the verifier's own docstring and the *coupling* note in `issuer_reenc`). A colluding issuer+recipient can therefore still defeat naming: address `E_note` to the recipient's *real* key (so the note is spendable) while encrypting `eIss` under a *throwaway* key. Every leaf then carries a valid binding (the leaf-tie and `verifyIssuerReenc` both pass), the recipient spends, yet under compulsion their `sk_rec` decrypts `eIss` to a meaningless point – no receipt names the issuer. This is a genuine residual hole in the invariant's third (collusion) clause, **not** closed by Phase 2c. See *The A2 recipient-key coupling gap*.

12.4 The A2 recipient-key coupling gap

The leaf-tie and the count check both concern *which committed leaf a binding answers to*. The remaining gap is orthogonal: *whether the key eIss is encrypted under is the addressed recipient's*. Nothing on chain forces it.

Attack (collusion, survives Phase 2c). Issuer I and recipient D collude.

- `E_note` := re-randomisation of D's REGISTERED credential (D can spend)
- `eIss` := `ElGamal(M_I^reg, pk_bogus)` for a key D does NOT hold
- `idHash` := `Poseidon8(E_note, eIss)`; `cm` := `Poseidon5(A2, v, rho, idHash, 0)`

`mint_batch_a2` proves `flavor=A2`, `idHash` opening, exposes `eIss`; `Notes.mint` field-ties `eIss` and `verifyIssuerReenc` accepts (`eIss` DOES re-encrypt `M_I^reg`, just under `pk_bogus` -- the issuer need not know `pk_bogus`'s discrete log). D spends via `spendACP` (`E_note` under D's real key). Compelled, D decrypts

eIss with sk_D -> garbage. No receipt names I. Untraceable A2 payment.

The structural cause: **E_note** and **eIss** are *independent* ciphertexts, both folded only into **idHash**; the recipient's key enters the system at **spend** (the **spendACP** CP-DLEQ ties **E_note** to the spender's registered key) but **eIss**'s key is never tied to **E_note**'s. The honest-recipient *coupling* in the receipt verifier (**verify_receipt**, note-a2) recovers M_{iss} and proves $pk_{rec} = Q$'s key – but that is *completeness* (an honest D *can* name I); it does nothing against a D who simply declines, which is the collusion clause.

The fix is the in-SNARK E_note ↔ eIss key-coupling. **E_note** cannot be exposed on chain (it is revealed at spend, so exposing it at mint would link mint to spend), so the coupling must live where **E_note** is already a private witness: inside **mint_batch_a2**. Add to each A2 leaf a witness $(sk_{iss}, pk_{rec}, r', r_n)$ with the issuer's registered $(R_{iss}^{reg}, C_{iss}^{reg})$ a public input (the issuer is **msg.sender**, so their credential is public), and constrain

$$pk_{iss} = sk_{iss}G, \quad M_{iss} = C_{iss}^{reg} - sk_{iss}R_{iss}^{reg}, \quad E_{iss} = (r'G, M_{iss} + r'pk_{rec}), \quad E_{note} = (r_nG, M_{rec} + r_npk_{rec}),$$

the *same* pk_{rec} in both.

Correction (found while scoping the phase): the shared-key constraint above is NOT sufficient by itself – it is vacuous. M_{rec} (and, with it, the recipient credential) is a *free* witness, so for a bogus note ($eIss$ under pk_{bogus} , E_{note} really under the recipient's pk_D) the issuer witnesses $pk_{rec} = pk_{bogus}$ and $M_{rec} = M_D + r_n(pk_D - pk_{bogus})$: both equations hold, the circuit accepts, the note still spends. The free recipient-side witness absorbs the constraint. A *sound* coupling must **pin pk_{rec} to a real registered key**, which forces an in-circuit **registry set-membership**:

1. witness the recipient's registered $(pk_D, E_{addr}[D])$ + a Merkle path proving the leaf $Poseidon(pk_D, E_{addr}[D])$ is in the IdentityRegistry accumulator (root a public input) – so the credential is *real*, not invented;
2. constrain $E_{note} = \text{rerand}(E_{addr}[D])$ under pk_D (so E_{note} is genuinely under that registered key – and hence spendable only by D, which **verifySpendCP** re-confirms at spend);
3. constrain $eIss = (r'G, M_{iss} + r'pk_D)$ under the *same* pk_D .

Now the spender is provably the holder of the key $eIss$ is encrypted under, so a compelled sk_D *must* recover M_{iss} . The membership proof reveals only that the recipient is *some* registered account (the leaf is a private witness; only the root is public), so A2 recipient privacy survives.

This is materially heavier than "a handful of G1 muls". The identities live on BN254 $\mathbb{G}_1$ – the proof system's own curve – so every point op (the rerandomisation, the two encryptions, $sk_{iss}R_{iss}$) is *non-native* $\mathbb{F}_p$ arithmetic emulated in an $\mathbb{F}_r$ circuit (circom-pairing / circom-ecdsa style), realistically $\sim 10^6$ R1CS **per A2 leaf**, plus the new registry accumulator. This is the recommended next phase for A2 – *not* shipped; Phase 2c deliberately stops at the leaf-tie (cheap, and a strict prerequisite: the coupling needs the per-leaf **eIss** exposure it adds). It cannot be moved on-chain: **E_note** is never on chain at mint (exposing it links mint to spend), so the one venue where **E_note**, **eIss**, and pk_{rec} coexist privately is inside **mint_batch_a2**. See *Scope: the A2 coupling phase*.

12.4.1 Can it be closed now? The non-native-curve obstacle

The cost line above ("a handful of $\mathbb{G}_1$ scalar-muls") understates the real obstacle, which is *which curve those muls are on*. The identities live on **BN254** $\mathbb{G}_1$ – the same curve the Groth16 proof system is built over. A circom circuit's native field is BN254's *scalar* field r ; BN254 $\mathbb{G}_1$ coordinates

live in the *base* field $p \neq r$. So every $\mathbb{G}_1$ operation in the coupling ($sk_{iss}R_{iss}, r'pk_{rec}, r_npk_{rec}$, the point adds) is *non-native* $\mathbb{F}_p$ arithmetic emulated inside an $\mathbb{F}_r$ circuit – limb decomposition + `mulmod` with range checks, the circom-pairing / circom-ecdsa style. circomlib’s EC gadgets are **BabyJubjub** (whose base field *is* r , hence native and cheap), not BN254 $\mathbb{G}_1$; there is no drop-in. Each emulated $\mathbb{F}_p$ multiply is thousands of $\mathbb{F}_r$ constraints, so the coupling is realistically $10^5 - 10^6$ R1CS *per A2 leaf* – pushing `mint_batch_a2` from $\sim 170K$ total to millions, a larger ρ (2^{22+}), and minute-to-hour prove times. This is exactly the native- $\mathbb{G}_1$ -in-circom cost the spend side already declined; it is a **dedicated multi-week phase**, not a now-fix.

The escapes all hit a wall:

- *Move identities to BabyJubjub* (native in-circuit EC): a ground-up redesign of the identity layer – the PS credential (a pairing), ElGamal, the registration NIZK, and the on-chain CP-DLEQ / Schnorr / re-enc verifiers all depend on BN254 + the EIP-196/197 precompiles, which BabyJubjub lacks.
- *On-chain sigma coupling* (cheap via EIP-196, like `verifyIssuerReenc`): needs `E_note` as an input, but `E_note` is the spend ciphertext – putting it (or a deterministic commitment to it) on chain at mint re-links mint to spend, the precise unlinkability we protect.
- *Spend-side enforcement* (prove at `spendACP` that sk_{rec} decrypts `eIss` to the issuer’s registered M): same non-native $\mathbb{G}_1$ in the *spend* circuit, and it needs γ (disclosed only in the compelled receipt) to know the target point – revealing it at spend would de-anonymise the issuer publicly.

12.4.2 The backstop: the gap degrades to KYC-traceability, not untraceability

The colluding A2 pair does *not* obtain untraceable money. Both parties act under on-chain addresses – the issuer is `msg.sender` at mint (and the funds pulled trace to them), the recipient is `msg.sender` at `spendACP` – and each address is a *registered* Identity whose KYC issuer (the trusted PS-credential signer) can de-anonymise it under subpoena. So the bogus-`eIss` attack costs the pair only the **bilateral, self-service receipt** (the recipient can no longer *unilaterally* name the payer); it does *not* defeat **KYC traceability** (a court compelling the registry’s trusted issuer still names both). The receipt invariant is the stronger, third-party-free guarantee; the gap means A2 *collusion* falls back to the weaker subpoena-the-KYC-issuer path that every registered Identity is subject to anyway. Whether to spend a multi-week non-native-EC phase to restore the stronger guarantee for the A2-collusion corner, or to accept the KYC backstop there, is a threat-model decision – it is **not** a silent hole: `verify_receipt` stamps such a note UNVERIFIED ISSUER.

12.4.3 Scope: the A2 coupling phase

The sound construction (set-membership + the three constraint groups above) is a **major cryptographic-engineering phase**, not an increment. Workstreams, in dependency order:

1. **Registry accumulator.** IdentityRegistry gains an incremental Poseidon Merkle tree of registered entries (leaf `Poseidon(addr, pk, E_addr)`), maintained on `register` / `bindContract`, with an on-chain root the mint circuit consumes. Decision: on-chain Poseidon (gas on each registration – rare, tolerable) vs an off-chain-computed root checked by a small SNARK. Mirrors the Notes pool’s tree but over the identity set.
2. **Non-native BN254 $\mathbb{G}_1$ gadgets.** Integrate a circom bigint / EC library (circom-pairing / circom-ecdsa lineage) for $\mathbb{F}_p$ arithmetic + variable-base scalar-mul + point-add over BN254

$\mathbb{G}_1$ inside the $\mathbb{F}_r$ circuit. This is the dominant cost and the dominant **audit** surface (under-constraint in emulated EC is the classic footgun – Picus/ecne the sub-templates). Handle the $\text{mod } r$ vs $\text{mod } p$ coordinate canonicalisation (the same care **eIss** < FIELD_R already takes, now load-bearing).

3. **Circuit mint_batch_a2_coupled.** Extend `mint_batch_a2` per leaf with the Merkle-membership + rerandomisation + dual-encryption constraints; new public inputs (**registryRoot**, the issuer’s registered E^{reg}). The **eIss** leaf-tie and Merkle insertion stay.
4. **Ceremony + sizing.** At $\sim 10^6$ R1CS/leaf the practical batch sizes **shrink** ($N=1,2,4$; $N=16/32$ would need **pot25+**, a multi-day, hundreds-of-GB ceremony). Expect a separate small- N pin set and a fresh, larger ptau .
5. **Verifier + Notes wiring.** New per- N `MintBatchA2CoupledN*Verifier` + adapter (arity grows by the new public inputs); **Notes** reads the registry root and routes the A2 path to the coupled verifier.
6. **Wallet prover + tests.** The prover supplies the recipient credential + Merkle path + rerand/encryption witnesses (real ciphertexts now, not arbitrary words); a **collusion regression** (**eIss** under a non-recipient key now **reverts at mint**) and a **privacy** test (two mints to distinct recipients indistinguishable). `verify_receipt` drops the UNVERIFIED ISSUER path for coupled mints.

Cost / risk envelope. Dominant: non-native EC – $\sim 10^6$ R1CS/leaf, minute-scale proves, big ceremony, heavy audit. Plus a new registry-accumulator subsystem. Realistic effort: a *design spike* (accumulator shape; pick/port the EC library; pin the exact relation) -> an $N=1$ *proof-of-concept* (witness-check the collusion + privacy properties end to end) before committing to the per- N ceremony + wiring + audit. A backend with native non-native support (Halo2 / PLONK lookups, or a 2-cycle like BN254/Grumpkin for native in-circuit EC) would cut the EC cost dramatically but is itself a proving-stack migration.

Where the cost lands – and a substrate that absorbs it. The $\sim 10^6$ R1CS is a *prover* cost; the on-chain Groth16 *verification* is constant ($\sim 250K$ gas) whatever the circuit size, so EVM cost was never the obstacle. Since the wallet proves off chain, a free-compute substrate – *Holochain*, which already holds the wallet’s bulk data – absorbs it: the agent hosts the registry tree (only the *root* need be on-chain), generates the non-native-EC proof at leisure, and the EVM does one constant-gas verify. Per-leaf proving with recursive aggregation (one verify for many leaves) sidesteps the hundreds-of-GB batch ceremony and restores the larger N the line above gives up. The envelope thus shifts: the *prover-feasibility* objection (minute-scale proves, giant ceremony) largely dissolves on such a substrate, leaving the proving-stack choice (thread 2) and – the true dominant residual – the **audit**, which free compute does nothing for.

Recommendation. Weigh this against the KYC backstop (above) before starting: it restores *bilateral, self-service* non-deniability for the A2-collusion corner only, at the price of a major phase and shrunken A2 batch sizes. If greenlit, gate it on the spike + PoC; do not begin the ceremony/wiring/audit until the $N=1$ PoC confirms the relation is sound and the costs are as modelled.

12.4.4 The identity-targeted alternative: name the identity, not the account

The account-targeted coupling above is hard because it fights the grain of the system. A direct EOA transfer *deduces* the identities from the two accounts in play – the account is primary, M is derived. A Note is the opposite: it *names* the identities and treats the accounts as incidental

envelopes, required only to *match* the expected M . The account-targeted construction re-imports the EOA model (pin a specific registered *account* pk_D) and pays for it in non-native account-leaf membership. Targeting the *identity* directly is the lighter problem, and it is the one that fits the spirit of Notes.

The shift. Encrypt the issuer payload under the recipient's identity *point* rather than an account key:

$$eIss = (R_e, C_e) = (rG, M_I + rM_{\text{rec}}), \quad M_{\text{rec}} = m_{\text{rec}}G,$$

where $m_{\text{rec}} = H(\text{id}_{\text{rec}})$ is the identity scalar that *every* Fountain account of the recipient shares. The decryption secret is therefore m_{rec} , not any account key: the issuer needs only M_{rec} (which the recipient revealed off chain), and *any* account the depositor later uses can open $eIss$. This is exactly "the spender knows the recipient's M , not which account will deposit."

Anti-framing stays at mint (shipped). `verifyIssuerReenc` already forces $eIss$ to re-encrypt `msg.sender`'s *own* registered M_I under the committed point, so the decrypted issuer is always the true minter, never a substituted victim.

The coupling moves to spend, as a registry-identity membership. Maintain a registry accumulator of registered identity *points* M with public root rt_{id} . The depositor's spend circuit

$$(N1) \quad M_{\text{rec}} = m_{\text{rec}}G$$

$$(N2) \quad \text{idHash} = \text{Poseidon}(M_{\text{rec}})$$

$$(N3) \quad M_I = C_e - m_{\text{rec}}R_e$$

$$(N4) \quad 1 = \text{MerkleVerify}(\text{rt}_{\text{id}}, M_I)$$

proves, with private witnesses $(m_{\text{rec}}, M_I, \pi)$ and the committed $eIss = (R_e, C_e)$: \[

\] The receiving account is separately tied to M_{rec} by the existing on-chain `verifySpendCP` (so the KYC-identifiable payout vehicle matches the named identity).

Why it closes the gap. (N3)+(N4) are *mandatory* to spend. A colluding issuer who keys $eIss$ to anything but M_{rec} makes (N3) decrypt to a non-member, (N4) fails, and the **spend reverts** – there is no spendable-but-unnameable note. (N1)+(N2) stop the depositor cheating by feeding a convenient m_{rec} : it must be the scalar of *this note's committed target*. Everything is in zero knowledge, so M_I and M_{rec} never appear on chain – this *overcomes the earlier spend-side objection* (*Can it be closed now?*), which assumed M_I had to be revealed; a membership proof of the decrypted point reveals nothing.

The receipt is unilaterally recipient-produceable. The depositor holds the one secret m_{rec} that names *both* parties: their own identity $M_{\text{rec}} = m_{\text{rec}}G$ trivially, and the issuer's $M_I = C_e - m_{\text{rec}}R_e$ by decryption. So even "in a pinch" – with no issuer participation at spend – the recipient can produce a plaintext receipt naming both, third-party-checkable against rt_{id} and the mint `verifyIssuerReenc`. This is the *recipient-burden* posture made cryptographically complete rather than merely economic: the always-KYC-identifiable party holds the entire bilateral receipt, and the mandatory (N3)+(N4) make producing it a *precondition of payment*.

Why it is the easier problem. The expensive account-set-membership over full account leaves $\text{Poseidon}(pk_D, E_{\text{addr}}[D])$ is replaced by a *native* Poseidon-Merkle membership of a single point M_I ; the non-native EC collapses to two scalar-muls ($m_{\text{rec}}G$ and $m_{\text{rec}}R_e$) against the account-targeted construction's five-odd (a rerandomisation plus two full encryptions plus $sk_I R_I$). A further split is open: prove (N1),(N3) as an *on-chain blinded sigma* (EIP-196, the `verifyIssuerReenc` shape, values Pedersen-committed and hidden) and leave *only* the native membership (N4) to a small SNARK – removing non-native EC from the circuit entirely. Residuals are honest: the depositor's own-identity tie (fold into `verifySpendCP` vs. an in-circuit check), the identity accumulator's root maintenance (Holochain holds the tree; only the root is on-chain), and a full cost + audit. It is a **promising variant to PoC**, not a shipped result – but it is materially lighter than the account-targeted phase and matches what a Note *is*.

12.5 Cost and migration

The added constraints are a handful of equality checks plus the `issuerMode` range/flavor gate per leaf – cheap relative to the dual Merkle walk already dominating the circuit (measured: +6 R1CS/leaf for the `issuerMode` gate). Any circuit change re-runs the per-N trusted setup and redeploys the per-N verifiers, rotated in via `Notes.setMintVerifier` behind a governance cutover root (the migration story in *Phasing*). The `issuerMode` regen was carried out this way (reusing the existing `ptau`, regenerating `MintBatchN{1,2,4,8,16,32}Groth16Verifier`). The A2 `eIss` leaf-tie does not touch `mint_batch` at all – it ships as the `separate mint_batch_a2` family (its own per-N verifiers + ceremony), so the existing public/bearer verifiers stay byte-for-byte as deployed. The public-leaf M tie needs no circuit (off-chain, above). The relation the tie enforces is already shipped as the off-chain `verifyIssuerReenc` binding; `mint_batch_a2` only **exposes** `eIss` so `Notes.mint` can field-match it.

DRAFT

13 The EOA Transfer Mirror: Anonymity Is the Cost

A standard private->private EOA transfer achieves the *same* mutual-decryptability invariant with **no SNARK** and **free** collusion-resistance. Splitting that transfer into `Notes.mint` -> `Notes.spend` through an A2 note is what makes it expensive. Mapping the cryptographic steps of the two flows isolates exactly *why*, exactly *what it buys*, and – decisively – *where the cost comes from*. The conclusion reframes the A2 coupling phase (*The A2 recipient-key coupling gap, Scope*) from "necessary work" to "an optional premium", and sets the shipping priority: **public-issuer notes (A1/B1) now, A2 reserved as a future option**.

13.1 The private->private EOA transfer (Alice -> Bob)

Both parties are encrypted-Identity; the transfer is *not* anonymous (the Alice->Bob link is on chain). Mutual decryptability is the bilateral **approve** handshake plus a compelled verifiable decryption.

Step	Cryptographic step	Who is named
E1 register	store $(pk, E_{\text{addr}} = \text{Enc}(M, pk))$ + PS credential	–
E2 approve (bilateral)	verifyApprove : $E_{A \rightarrow B} = \text{Enc}(M_A, pk_B) + \text{Chaum-Pedersen } E_{A \rightarrow B} \equiv E_{\text{addr}}[A]$	both (reads $E_{\text{addr}}[A], pk_A, pk_B$)
E3 transfer	value Alice->Bob, gated on the approve	both (link visible)
E4 receipt	verifiable_decrypt $E_{X \rightarrow \text{me}}$ -> reveal M	–

13.2 The A2 Note flow (Issuer I -> Recipient D)

Both parties are encrypted-Identity; the flow *is* anonymous (mint and spend are unlinkable). The issuer is present and named at mint; the recipient is present and named at spend; they are never co-present.

Step	Cryptographic step	Who is named
A1 register	identical to E1	–
A2 mint	issuer_reenc : $eIss = \text{Enc}(M_I, pk_D) + \text{CP } eIss \equiv E_{\text{addr}}[I]$ – but pk_D blinded ($Q = pk_D + \beta H$)	issuer only (reads $E_{\text{addr}}[I]$)
A3 spend	verifySpendCP : sk_D decrypts E_n & $E_{\text{addr}}[D]$ to the same M_D	recipient only (reads $E_{\text{addr}}[D]$)
A4 receipt	verifiable_decrypt $eIss$ -> M_I + the γ coupling	–

13.3 The mapping

EOA step	A2 step	Same relation?	The difference
E2 Alice->Bob approve (verifyApprove)	A2 mint issuer_reenc	yes	pk_D blinded vs
E2 Bob->Alice approve (sound, named)	A2: nothing sound – D's address revealed at spend + KYC	no	A2's issuer-name
E3 transfer (Alice<->Bob linked)	A2 mint->spend (unlinked)	no	A2 adds bulk-sur
E4 verifiable_decrypt	A4 verifiable_decrypt + γ coupling	yes (same prim.)	A2 adds the blin

The first row is the whole story: **issuer_reenc** is **verifyApprove** with the recipient's key blinded. The CP relation, **verifiable_decrypt**, and the receipt are identical; the one change – blinding pk_D – is the entire gap.

13.4 The structural diagnosis: the two anchors are never co-present

`_verifyApprove` is cheap *and* collusion-resistant because it reads **both** parties' registered records in one call (`_E_addr[sender]`, `_pk[sender]`, `_pk[spender]`) – both anchors co-present, both named. In A2 the anchors are *split in time by anonymity*:

- the **issuer's** anchor $E_{\text{addr}}[I]$ exists only at **mint** ($I = \text{msg.sender}$);
- the **recipient's** anchor $pk_D, E_{\text{addr}}[D]$ exists only at **spend** ($D = \text{msg.sender}$).

No on-chain check can read both at once the way `verifyApprove` does, so `issuer_reenc` blinds pk_D to bridge them – and blinding pk_D is exactly what drops the "under the recipient's real key" guarantee. The set-membership SNARK (*Scope*) is the **only** venue that re-unites the anchors ($E_{addr}[I]$ a public input at mint, pk_D a membership-proven witness). *That separation, and the cost of bridging it, is the whole reason A2 is hard.*

13.5 The finding: the cost is the anonymity

EOA's mutual decryptability needs no SNARK and is collusion-resistant *for free* precisely because the transfer is **not anonymous** – both parties are named at one moment. A2's entire extra apparatus (the mint SNARK, the pk_D blinding, the coupling gap, and the set-membership phase that would close it) is the price of **mint $\leftrightarrow$ spend unlinkability**. There is no free lunch: cheap collusion-resistant mutual decryptability *requires* naming both parties; anonymity *forbids* it. Three corollaries:

1. **The cost is the anonymity** (above) – the irreducible tension.
2. **A2 is already asymmetric.** The recipient-names-issuer direction is the *only* one A2 holds to "collusion-resistant, on-chain, self-provable"; the issuer-names-recipient direction is merely "D's registered address is on chain at spend + KYC" – the *same* KYC backstop, not collusion-resistant. The coupling phase would upgrade one direction to a bar the other does not meet.
3. **B-notes already show the cheap path.** `verifyDepositorForIssuer` is *literally* `verifyApprove`, cheap and collusion-resistant for B-notes **because the issuer is public (named)**. So A2's hardness is specifically *issuer anonymity*. The system already offers the cheap, fully-sound option: mint as a **public issuer (A1/B1)** $\rightarrow$ plain `verifyApprove` / Schnorr, already shipped.

13.6 The recipient bears the burden – the load-bearing safeguard for A2

This is what makes A2 shippable as a future option *without* the coupling, and it sharpens (and lightly corrects) the operator intuition that "depositing an A2 note exposes the recipient."

An *honest* recipient verifies the note **at delivery**, off chain, before accepting it: decrypt $eIss$ with sk_D , and check it yields the issuer's **registered Identity** (run the γ coupling). A bogus note ($eIss$ under a throwaway key, or γ withheld) **fails this check**, so an honest recipient **rejects the payment** – they will not accept value they cannot later receipt. Therefore a recipient who *accepted and deposited* a note that later proves un-nameable (UNVERIFIED ISSUER under compulsion) was either **negligent** (skipped the delivery check) or **colluding**. Either way, **the suspicion falls on the recipient, not on the (anonymous) issuer**. The recipient is always KYC-identifiable – their *own* registered address is `msg.sender` at spend – so under audit they must produce a valid issuer-naming receipt or stand as the holder of funds from an unaccountable source.

Correction to the framing: the collusion cannot **falsify** an issuer identity (frame an innocent third party). *Receipt soundness / non-frameability* (Theorem above) prevents binding $eIss$ over a victim's credential – the binding is always over the *actual minter's* registered record, so the only reachable outcomes are "names the true minter" (VALID) or "names no one" (UNVERIFIED ISSUER). Collusion can **deny / obscure** the issuer (render it un-nameable); it cannot **substitute** a false one. And the flag is not automatic on chain – the spend reveals no $eIss$ – it surfaces *at audit or compulsion*, when the recipient is asked to account for the deposit.

The consequence is a clean economic posture: A2 does not give cryptographic *bilateral* non-deniability under collusion, but (a) the recipient is always KYC-identifiable; (b) the recipient is

incentivised to verify at delivery (the honest recipient is fully protected); and (c) accepting an unnameable A2 note is a **self-inflicted AML red flag on the colluding recipient's own KYC'd account**. The collusion risk is *borne by, and self-limiting to, the recipient* – which is why the on-chain coupling can be deferred without leaving an unbounded hole.

13.7 Decision: A1/B1 now, A2 reserved

- **Ship public-issuer notes (A1, B1) as the primary product.** They already carry the full EOA-grade guarantee – sound, collusion-resistant, on-chain, bilateral – via `verifyIssuerSchnorr` / `verifyDepositorForIssuer` (= the `verifyApprove` family). No SNARK coupling, nothing reserved.
- **Reserve A2 (private-issuer) notes as a future option**, gated by the recipient-burden posture above: an A2 deposit a recipient cannot receipt marks *the recipient*. Wallets **MUST** verify *eIss* at delivery and refuse a note that does not resolve to a registered issuer, and **SHOULD** surface, on the A2 receive screen, that accepting an unverifiable A2 note transfers the accountability to the recipient.
- **The set-membership coupling phase (*Scope*) is the premium** that would restore EOA-grade *bilateral* non-deniability to A2-under-collusion – to be undertaken only if a deployment specifically needs no-trusted-third-party traceability for private-issuer notes, and only after the design spike + N=1 PoC.

13.8 Directions for future research

The open problem, stated cleanly by the mirror: *can collusion-resistant mutual decryptability be achieved when one party is anonymous at the binding step (the recipient at A2 mint), without in-circuit set-membership?* The structural diagnosis suggests the anchors' time-separation forces ZK set-membership, but this is not proven. Concrete threads:

1. **Lower bound / impossibility.** Formalise "the two anchors are never co-present on chain" and ask whether *any* on-chain-only protocol (no SNARK over the registry) can bind *eIss* to the recipient's registered key while hiding which recipient. A clean impossibility would settle that the SNARK is necessary; a loophole would be a cheap fix.
2. **Proving-stack choice for the set-membership route.** Non-native BN254 $\mathbb{G}_1$ in circom/Groth16 is $\sim 10^6$ R1CS/leaf. A backend with native non-native support – Halo2/PLONK with lookups, or a 2-cycle (BN254/Grumpkin) giving *native* in-circuit EC on the partner curve – could cut this by an order of magnitude. Evaluate vs the Groth16 toolchain already in the repo. Orthogonally to the *circuit* cost, the *prover* cost is off-loadable: the proof is generated off chain on the wallet's Holochain agent (which already holds the registry tree and has ample compute) and the EVM verifies it in constant gas – so the stack choice and the substrate compose, a cheaper stack shrinking the circuit and a free substrate absorbing whatever remains.
3. **Registry accumulator design.** On-chain incremental Poseidon tree (gas on each registration) vs an off-chain-maintained root proven correct by a small SNARK; interaction with the existing mapping-based registry.

4. **Recipient co-participation, and where the binding lives.** The account-targeted coupling (*Scope*) does *not* actually need the recipient online: the issuer witnesses the recipient's *public* registry record ($pk_D, E_{\text{addr}}[D]$, Merkle path) – it is addressing the note to that account – and the in-SNARK membership needs no sk_D . So the issuer mints to an *offline* recipient. Two richer variants deserve a rigorous treatment:

- **Interactive A2.** If both parties are co-present off chain (Holochain), they run the full bilateral **approve** inside the wallet and prove it at mint – the EOA transfer's *free* collusion-resistant mutual decryptability – while still depositing later, unlinkably. This buys EOA-grade bilateral non-deniability at the cost of an online recipient (fine for self-wormholes and present counterparties).
- **Identity-targeted, spend-side membership.** Address the note to the recipient's *identity point* M_{rec} rather than a specific account, encrypting $eIss$ under M_{rec} (decryptable by the identity scalar m_{rec} that *every* Fountain account of the recipient shares) – so the issuer needs only M_{rec} , never an account. Enforce the coupling at *spend*: the depositor's circuit *mandatorily* proves " $eIss$, decrypted under the identity m I authenticate as, yields M_I , and $M_I \in$ the registry-identity tree" – in ZK, so M_I stays hidden. A bogus $eIss$ decrypts to a non-member and the spend *reverts*; a colluding recipient cannot both spend and decline to name. This *overcomes the earlier spend-side objection* (*Can it be closed now?*): that objection assumed M_I had to be *revealed* at spend, but a ZK membership proof of the decrypted point reveals nothing. It needs one non-native scalar-mul (the decryption) plus a *native* Poseidon-Merkle membership against a tree the Holochain agent already holds – plausibly cheaper than the mint-side account-targeted coupling, and the recipient's m_{rec} is present at spend by construction. Anti-*framing* still rides on the mint-side **verifyIssuerReenc** (it forces $eIss$ over the issuer's *own* registered M_I , so the spend-decrypted member is the true minter, never a substituted victim). Open: a full circuit cost + audit, and whether the depositor's own identity proof folds into the existing **verifySpendCP** or needs an in-circuit credential check. Full treatment with the spend relation in display math: *The identity-targeted alternative: name the identity, not the account.*

5. **Mechanism design as an alternative to crypto.** Formalise the "recipient bears the burden" safeguard: under what audit probability and penalty does the recipient-burden deter A2 collusion as effectively as cryptographic prevention? If the economic argument is tight, it *obviates* the coupling phase – the most leverage for the least engineering.
6. **Auditor backstop stronger than subpoena.** Threshold/selective escrow of γ (or of M_I) to a designated regulator key, so A2 collusion is detectable by an auditor without public disclosure – a middle point between "bilateral, no third party" and "subpoena the KYC issuer."

14 Implementation Status (Phase 1 + Phase 2 A2 binding + issuerMode + A2 leaf-tie shipped)

This section records what is **actually shipped** against the spec above, so the state is self-contained. Work proceeds on branch `feature/decryptability` (cut from `master` after the `feature/buck-negative-balance` merge, `f445905`).

14.1 Phase 1 – shipped, wired, and tested

The public-issuer half and the bearer depositor->issuer half are implemented end-to-end. Commits, in order:

- `d8dc818` – `IdentityRegistry.verifyIssuerSchnorr + SchnorrProof` struct + `_fsIssuerSchnorr` helper. Checks $s * G = R + e * pk_{iss}$, gated on `isPublicIdentity[issuer]`; Fiat-Shamir binds $(pk_{iss}, R, hBatch, issuer, chainid)$.
- `0df3e71` – `Notes.mint` split into a 6-arg (legacy) and 7-arg (+ `SchnorrProof`) overload over an internal `_mint`. When `isPublicIdentity(msg.sender)`, `_mint` requires `verifyIssuerSchnorr(msg.sender, keccak256(cms), sig)`; the 6-arg path passes a zero proof and so reverts for a public issuer. New `IssuerBound(issuer, newRoot)` event.
- `5f92cdc` – `wallet/alberta_buck/wallet/schnorr.py` (`issuer_schnorr_sign/verify, batch_commitment` = keccak of packed cms); `vectors.py` emits an `issuer_schnorr` section into `test/vectors/identity.json`; the FS transcript matches `_fsIssuerSchnorr` byte-for-byte.
- `0f0ab0f` – `IdentityRegistry.verifyDepositorForIssuer` (extracted the `verifyApprove` body into `_verifyApprove`; the alias just relabels sender/spender as depositor/issuer). `Notes.spend` split into a 5-arg (legacy) and 8-arg (+ `issuer, eDepForIss, CPProof`) overload over `_spend`; the 8-arg path checks `verifyDepositorForIssuer(recipient, issuer, ...)` and emits `SpentB`.
- `2f23dae` – `RcptVerify`, the off-chain receipt verifier (`alberta_buck/wallet/receipt.py`): a `Receipt` (opening + minted batch + issuer Schnorr + Spent-event nullifier/face) and `receipt_verify` that checks the four-link chain for **public-issuer** notes (A1, B1) and names the issuer's registered Identity, refusing A2 (private issuer, Phase 2). No secret is needed to verify, so a third party can re-check it. A canonical `receipt` section was added to `test/vectors/identity.json` (Bob, a public Corporate Identity, mints a B1 bearer note Alice cashes). This is the Phase 1 capstone: links (a),(b),(d) were already sound on-chain and (c) is now bound for public issuers, so the receipt is finally **produceable**.
- `2f23dae` – the symmetric **direct EOA-transfer** receipt. A new `alberta_buck/wallet/verifiable_decrypt` primitive (a Chaum-Pedersen DLEQ on bases (G, R) , the `spend_cp` shape specialised to $(R, C-M)$) proves an ElGamal ciphertext decrypts to a **revealed** M under a registered key without exposing sk – the "no deniability under collusion" clause for an encrypted-identity counterparty. `receipt.py` gains `ApproveReceipt + approve_receipt_verify`, composing the sender's approve handshake (`chaum_pedersen_verify / verifyApprove`, soundness) with the recipient's verifiable decryption (recovery + provability) to name the sender's registered Identity, third-party-checkable with no secret. This packages what the wallet previously left a la carte (`chaum_pedersen + elgamal_decrypt`) and shares `RegisteredIdentity / RcptResult` with the Notes verifier. An `approve_receipt` section (Bob names Alice from the Alice->Bob

approve) is added to `identity.json`. No on-chain verifier ships (off-chain, like `RcptVerify`); the verifiable-decryption primitive is also the building block the A2 Notes receipt will reuse (decrypt `E_iss-for-rec`, prove correctness).

- **9b9a268** – the AB-RCPT/1 **receipt core** + **ASCII renderer**: `envelope.py` (the deterministic `ReceiptCore`, canonical-JSON serialization, the text envelope, `receipt_id`), `build_receipt.py` (five scenario builders), `verify_receipt.py` (tier-1 offline verifier), and `render.py` (the extensible `Detail` / `ReceiptDoc` / `TextDriver` renderer). Plus the `alberta-buck-receipt.org` spec. This is the end-user artifact – the bit-identical, independently re-verifiable record + a printable slip.

Tests – all green at the close of Phase 1; the running totals after Phase 2 below are **forge 393 passed, 1 skipped** and **wallet pytest ~157 passed** (the offline-skipped `BuckKControllerForkTest`). Phase 1 suites:

- `test/IssuerSchnorr.t.sol` – 7 in-Solidity (completeness + soundness: tampered batch / response, wrong key, non-public, unregistered, cross-issuer replay) + 4 `IssuerSchnorrVectorTest` (the python-emitted proof verifies on-chain – the cross-artifact parity check).
- `test/NotesIssuerBinding.t.sol` – 5 (valid binding mints; tampered / wrong-batch sig and the 6-arg overload reverts `Notes: bad issuer binding`; non-public minter skips).
- `test/NotesDepositorBinding.t.sol` – 5 (registry parity reusing the `approve` vector as a depositor->issuer proof; bound spend emits `SpentB`; bad CP reverts `Notes: bad depositor binding`; legacy 5-arg unbound works).
- `alberta_buck/test/test_schnorr.py` – 6 (round-trip, tamper, keccak parity).
- `alberta_buck/test/test_receipt.py` – 17 (Notes: completeness names Bob’s registered M; third-party recheck needs no secret; soundness: tampered opening, unregistered / non-public / wrong-key issuer, forged batch sig, mismatched nullifier or face all reject; A2 refused. EOA ApproveReceipt: completeness names Alice; third-party recheck; frame-wrong-M, tampered cp/vd proof, substituted sender record, unregistered sender all reject).
- `alberta_buck/test/test_verifiable_decrypt.py` – 6 (round-trip; wrong M, wrong key, tampered response, replayed account/chain reject; the real key still cannot name a false M).

14.2 Phase 2 – A2 private-issuer binding (shipped, off-chain verifier + on-chain anchor)

The A2 issuer half – the one flavor where both parties are private – is now bound, via the **blinded-CP** route (Open Question 1, decided: option 2, not in-SNARK). Commits, in order:

- **5a9d0af** – wallet `alberta_buck/wallet/issuer_reenc.py`: the recipient-blinded re-encryption proof. A 5-relation Okamoto sigma; the linearisation publishes $U = r'H$ and $\hat{T} = r'pk_{rec} + \gamma G$ with $Q = pk_{rec} + \beta H$, so $r'Q - \beta U = r'pk_{rec}$ holds for any β once U is pinned – no product gadget, no pairing. 10 tests (completeness; the colluding-issuer attack – random/inconsistent leaf rejected; tamper; wrong key; replay; recipient-hiding).
- **1a03c5b** – `IdentityRegistry.verifyIssuerReenc` + `IssuerReencProof` + `_fsIssuerReenc`, the on-chain mirror (~111K gas, EIP-196). `test/IssuerReenc.t.sol` – 6 (cross-artifact parity + soundness).

- d000e40 – **the privacy fix**. The first draft published $T = r'pk_{rec}$, which leaks $M_{iss} = C_i - T$ (and `msg.sender` is the issuer at mint). Blind it: $\hat{T} = r'pk_{rec} + \gamma G$, carry γ through L3/L5 (witness γ , response `s_g`). Still sound – γ cancels – but M_{iss} and pk_{rec} are now hidden on chain. See *Data Revealed Per Step* for the worked argument.
- b5e8b79 – the A2 receipt coupling. `build_note_a2` carries the binding + the disclosed γ ; `verify_receipt` composes `verifyIssuerReenc` with the recipient's verifiable decryption and requires $M_{iss}^{reg} = C_i - \hat{T} + \gamma G = M_{named}$ – which algebraically forces $pk_{rec} = Q$'s key, the `E_note` $\leftrightarrow$ `Q` coupling, with no new proof. The A2 receipt now reads `VALID` (was `UNVERIFIED ISSUER`).
- 44a1363 – `Notes.mint(..., A2Binding[])`: verifies + anchors the A2 bindings on chain via `verifyIssuerReenc` (invalid binding reverts the mint), emitting `IssuerReencBound`. `test/NotesA2Binding.t.sol` – 4.

Decided (was Open Question 1): the **blinded-CP on-chain** route, not in-SNARK – it reuses the verified `verifyApprove` shape, costs $\sim 111K$ gas vs the $\sim +250K$ R1CS/leaf the spend side already declined for native G1 in circom, and needs no trusted-setup regeneration for the *relation*. The one in-SNARK change still required is the `issuerMode` + leaf-tie signal (see *The Required Mint SNARK Signal*), which is enforcement plumbing, not the cryptographic binding.

14.3 Phase 2b – per-leaf issuerMode bound in the mint SNARK (shipped)

The `issuerMode` half of *The Required Mint SNARK Signal* is shipped, making the bearer $\Rightarrow$ public-issuer gate **inescapable** (the on-chain image of *Why B2 is impossible*). Commits, in order:

- b14e0f0 – `mint_batch.circom` range-constrains `flavor` \in `{A1,A2,B1}` and emits the per-leaf public output `issuerMode` (`A2` $\rightarrow$ `PRIVATE`, `{A1,B1}` $\rightarrow$ `PUBLIC`). Witness-checked at `N=2`: `flavor[A1,A2]=` $\rightarrow$ `issuerMode[1,2]=`; `[B1,B1]` $\rightarrow$ `[1,1]` (bearer always `PUBLIC`); a would-be `flavor=4` ("B2") is unsatisfiable (+6 R1CS/leaf).
- 584bb52 – `Notes.mint` per-leaf gate: a `PUBLIC`-mode overload (Schnorr, registered public `msg.sender`) and a `PRIVATE`-mode overload (A2 bindings, one per private leaf, count completeness); mixed-mode / bad-mode / length mismatch revert. `test/NotesIssuerMode.t.sol` – 14, including the bearer-from-non-public reverts.
- 68e5b55 – regenerate `MintBatchN{1,2,4,8,16,32}Groth16Verifier` (reusing the existing `ptau`) so the Groth16 proof **binds** `issuerMode` (public-signal vector `[issuerMode[0..N], oldRoot, newRoot, nextLeafIndex, totalFace, cm[0..N]]`, arity `2N+4`); `IMintVerifier` / `MintVerifierAdapter` / `Notes` thread it; `prove_mint_batch.js` emits it; new `gen_mint_fixtures.sh` regenerates every fixture. `test/MintVerifier.t.sol` gains `test_mint_rejectedOnTamperedIssuerMode` (a flipped leaf mode fails the pairing check). Forge 408 passed, 1 skipped.

What this does **not** yet bind: the A2 `eIss` leaf-tie (completeness against the committed leaf set). That is Phase 2c, below; the public-leaf `M` tie was moved off-chain (see *The Required Mint SNARK Signal*).

14.4 Phase 2c – A2 eIss leaf-tie (SHIPPED)

Binds each committed A2 leaf's **eIss** to its own verified re-encryption binding – closing the count-only attack (a colluding pair floating duplicate bindings while committing distinct **eIss**; see *What the A2 tie buys*). Decisions: **raw eIss** exposure (not a Poseidon digest – on-chain hashing is EVM-hostile) in a **dedicated** `mint_batch_a2` circuit (so public/bearer batches pay nothing). Shipped:

- `f4c4060 – circuits/mint_batch_a2.circom`: the A2-only mint circuit. Per leaf it constrains `flavor = A2=`, opens `idHash` to `Poseidon8(eNote, eIss)`, and exposes `eIss = (R.x,R.y,C.x,C.y)` as a public output; `eNote` stays a private witness (mint $\leftrightarrow$ spend unlinkability). Public-signal layout [`eIss[0..N]`] (4 each), `oldRoot`, `newRoot`, `nextLeafIndex`, `totalFace`, `cm[0..N]`], arity `5N+4`.
- The per-N A2 ceremony (`scripts/snark/setup.sh render_mint_batch_a2_n + a MINT_BATCH_A2_PINS` loop, reusing the `mint_batch` `ptau`; `make snark-a2`) emits `src/MintBatchA2N{1,2,4,8,16,32}Groth16Veri`
- `IMintVerifierA2 + MintVerifierA2Adapter` (the `5N+4` `pubSignals`, `eIss` outputs leading) dispatch per-N by `cms.length`, mirroring `MintVerifierAdapter`.
- `Notes.mint` PRIVATE-mode overload now routes to `a2MintVerifier` (new slot + `setA2MintVerifier`), passing the bindings' `eIss` as the proof's public input: a Groth16 accept *is* the leaf-tie (the circuit exposes the committed leaf's `eIss`, so equality with what `Notes` passes is a SNARK constraint, not a contract-side compare – no on-chain Poseidon, no field loop beyond a canonical `eIss < FIELD_R` bound). Per-leaf `verifyIssuerReenc` still runs; `a2Bindings.length = cms.length=`.
- `prove_mint_batch_a2.js + gen_mint_fixtures_a2.sh` (a `--eiss` pin so a fixture leaf can carry the canonical `issuer_reenc` binding).
- Tests: `test/MintVerifierA2.t.sol` (cross-artifact parity; tampered `eIss` / `cm` fail the pairing check) and `test/NotesA2Tie.t.sol` (the leaf-tie mints against the *real* verifier; the **duplicate-binding** and **floating-binding** attacks revert `bad mint proof`). Forge 420 passed, 1 skipped.

What Phase 2c does NOT close: the recipient-key coupling (*The A2 recipient-key coupling gap*). The leaf-tie makes every committed leaf carry a verified binding; it does not force that binding's key to be the recipient's. Full A2 collusion-resistance is the in-SNARK `E_note $\leftrightarrow$ eIss=` coupling – the recommended next phase.

14.5 Decisions / deviations from the spec text above

- `verifyDepositorForIssuer` **reuses the verifyApprove relation verbatim** – no new sigma protocol, no new wallet signer, no new vector. The existing `approve` (Alice->Bob) vector **is** a valid `depositor(Alice)->issuer(Bob)` proof. FS binds (`depositor`, `issuer`, `chainid`); the specific note is tied in only by `SpentB` emitting (`nullifier`, `eDepForIss`) atomically – the `nullifier` is NOT in the CP transcript.
- `Notes.spend` **binds the recipient (the beneficiary), not `msg.sender`**.
- **mint is gated-only** – the two binding overloads (PUBLIC Schnorr, PRIVATE A2 leaf-tie) are the only mint paths; the legacy unbound 6-/7-arg overloads were removed. `spend` keeps the optional 5-arg (legacy) / 8-arg (bound B) forms: the `depositor->issuer` binding is a hardening, not load-bearing for the receipt (a B1 issuer is already named at mint).

- The shipped event is `SpentB(nullifier, face, recipient, issuer, eDepForIss)` (issuer included). The depositor **names** issuer as a parameter; it is **not** recovered from the leaf, because the B-spend circuit does not reveal it.

14.6 Closed since Phase 1

- **A2 re-encryption binding** – the central Phase 2 item. Shipped via the blinded-CP route (`verifyIssuerReenc`), *not* in-SNARK. Both the relation’s soundness and its issuer/recipient privacy are done and tested.
- **RcptVerify for A2** – the A2 receipt now verifies **VALID** (the coupling via `verify_receipt`, `b5e8b79`); the **UNVERIFIED ISSUER** banner remains only for a note whose issuer supplied *no* binding.

14.7 Residual gaps (NOT done)

1. **A2 recipient-key coupling** (the private-issuer collusion corner) – the account-targeted form (`eIss` keyed to a specific registered *account*) is bound to *a* key the issuer chose, so a colluding pair could leave the issuer un-nameable while the note stays spendable (*The A2 recipient-key coupling gap*). The leaf-tie (Phase 2c, **DONE**) and bearer-from-non-public (Phase 2b, **DONE**) are closed; the account-targeted closure remains the heavier set-membership + non-native- G_1 *premium*. **Update (2026-06-06): the lighter *identity-targeted* realisation is now shipped and bound** (*The identity-targeted alternative*; full treatment in *alberta-buck-notes-unilateral*). `Notes.spendCoupledA2` co-verifies the deposit-coupling sigma and a G_1 -tie membership proof, feeding the sigma’s committed point P_I to *both* – so a bogus `eIss` makes P_I ’s point a non-member and the deposit reverts. This closes the collusion corner for identity-targeted A2 notes (the membership is *native* Poseidon over a tree the registry already maintains, not the non-native-EC premium); where it is not used, the **recipient-burden** posture still bounds the gap (an un-nameable A2 deposit marks the KYC-identifiable recipient). Public-issuer notes (A1/B1) have no such gap.
2. **Issuer-in-the-B-spend circuit** – today the B-depositor *names* the issuer at `SpentB` (sound for an honest depositor); revealing the issuer from the leaf in the B-spend circuit would remove the honesty assumption. Lower priority – a B1 issuer is public and already named by `msg.sender` at mint, so the receipt does not depend on this; it only hardens the `SpentB` event.
3. **Formal verification** (Picus / Halmos / EasyCrypt) – not started.

Closed by gated-only minting: the "enforcement is opt-in" gap (the unbound 6-/7-arg `mint` overloads that let a non-public issuer append A1/B1/A2 leaves with no binding) is **gone** – those overloads, and the `bindingRequired` cutover flag that would have disabled them, were removed. `Notes.mint` now has exactly two paths (PUBLIC Schnorr, PRIVATE A2 leaf-tie), both requiring a registered issuer; "bearer => public issuer" and "every A2 leaf bound" hold by construction, not by the issuer’s overload choice.

15 Remaining Work and Approaches

Phase 2b (issuerMode) and Phase 2c (the A2 eIss leaf-tie) are **shipped**. The governing decision (*The EOA Transfer Mirror*) is: **public-issuer notes (A1/B1) are the shipping priority – they already carry the full EOA-grade guarantee – and A2 (private-issuer) notes are reserved as a future option under the recipient-burden posture**. In that light:

1. **Ship A1/B1 (priority)**. Nothing cryptographic is missing: `verifyIssuerSchnorr` (public-issuer batch binding) and `verifyDepositorForIssuer` (= `verifyApprove`) give sound, collusion-resistant, bilateral mutual decryptability for public issuers. Remaining work here is product, not crypto: wallet mint/receive/ receipt UX, indexer, and the end-to-end A1/B1 receipt path.
2. **A2 wallet posture (if A2 is offered at all)**. The recipient **MUST** verify `eIss` at delivery (decrypt with sk_D , run the γ coupling) and **refuse** a note that does not resolve to a registered issuer; the receive screen **SHOULD** warn that accepting an unverifiable A2 note transfers accountability to the recipient (*The recipient bears the burden*). This is the safeguard that lets A2 ship without the coupling.
3. **A2 recipient-key coupling (reserved premium)**. Restores EOA-grade *bilateral* non-deniability to A2-under-collusion; needed only if a deployment requires no-trusted-third-party traceability for private-issuer notes. The sound construction (set-membership + non-native $\mathbb{G}_1$), its cost, the research threads, and the spike+PoC gate are in *The A2 recipient-key coupling gap* and *Scope*. **Not on the critical path**.
4. **Enforce the eIss leaf-tie is reflected in the A2 receipt**. `verify_receipt` (note-a2) can now assert the spent leaf's `eIss` was leaf-tied at mint (the `UNVERIFIED ISSUER` path tightens to "issuer supplied no binding for *this leaf*"). Pure off-chain wallet work.

Off-chain (record for the wallet/receipt spec): the public-leaf issuer is named from `msg.sender` + the Schnorr batch binding, with the wallet locating the mint tx by scanning `Notes.mint calldata` for the `cm` (full chain history). The committed `m_issuer` is not used for naming.

Not in scope / deferred: the B-spend issuer-in-circuit (residual gap 3), formal verification (residual gap 4), and a production trusted-setup ceremony (the dev ptau + per-circuit contributions use fixed dev entropy – see [README.org](https://github.com/Albera-Buck/notes.org) "SNARK Circuits and Trusted Setup").

15.1 Cross-references and house conventions

- `alberta-buck-notes.org` and `alberta-buck-notes-flow.org` are kept current with this doc. Brought up to date alongside Phase 2b/2c: the stale `cmBatchHash` public-input (the shipped circuit exposes `cm[]` directly + `issuerMode[]`), the `mintVerifier` / `MintVerifierAdapter` dispatch, the per-leaf `issuerMode` gate, and the B2-impossibility framing (notes.org's "why bearer forces a public issuer"). Regenerated PDF/TXT exports may lag the `.org` masters.
- Conventions in force: a negative signed BUCK balance is **not** "debt" – it is "used credit" / an outstanding claim on the holder's own assets; the carrying transfer is the `isCarrying-dispatched Buck._carryingTransfer` (`buckSeconds` apportionment), **not** a `transferCarrying` primitive. Build and test via `make nix-test`; Python via `nix develop --command python -m pytest`. Commit messages: ASCII only, no co-author/AI attribution.

16 Open Questions

1. **In-SNARK vs blinded-CP for A2.** *Decided: blinded-CP* (`verifyIssuerReenc`, `5a9d0af / 1a03c5b / d000e40`). It keeps the mint SNARK out of native G1 arithmetic (the $\sim +250K$ R1CS/leaf the spend side already declined), costs $\sim 111K$ gas on chain, and reuses the audited `verifyApprove` shape; the bespoke part is the $U, \hat{T}, Q$ linearisation, written up and tested. The remaining circuit work (`issuerMode` + leaf-tie) is enforcement plumbing, not the binding relation.
2. **Batch vs per-leaf issuer signature.** A single Schnorr over `keccak(cms)` is cheapest but binds the *whole batch* to one public issuer; mixed-issuer batches need per-leaf signatures or per-issuer sub-batches.
3. **Receipt non-repudiation scope.** `RcptVerify` proves the issuer *authored* the note; tying that to a real-world legal identity still rests on the KYC binding of M (the PS credential). This document assumes that binding; it does not re-derive it.
4. **Predicate-bearing notes.** The `predicate` field is out of scope here; a programmable predicate that itself moves value would need its own receipt analysis.

DRAFT